Testarea Securității Sistemelor de Calcul

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

Foundation of Computer Security

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

Honor Code

"My job is to talk to you, and your job is to listen. If you finish first, please let me know."

Harry Hershfield



Lecture structure

- 10 lectures on selected topics:
  - Hardware Security
  - OS Security Design
  - Application Security
  - Forensics
  - Usable Security
  - Access Control
  - Cryptography
  - Network Security
  - Web Security

Lecture information

- http://ocw.cs.pub.ro/courses/isc

Grading

- **4p** - Written exam (TBD)
- **2p** - Portfolio
- **2p** - Homework 1
- **2p** - Lab (10 x 0.2)
- **Total = 10p**

What this lecture is **not** about

- Reverse engineering (go to http://ocw.cs.pub.ro/courses/cns)
- Detailed information on selected topic
- Hacking
- Security Certifications (Comptia Security+, CEH, CISSP, CISCO Security, etc.)

What this lecture is about

- An overview of what is out in the wild and how to protect against it
- A selection of tools interesting enough to be mentioned
- A focus on the academic component
- Security vocabulary

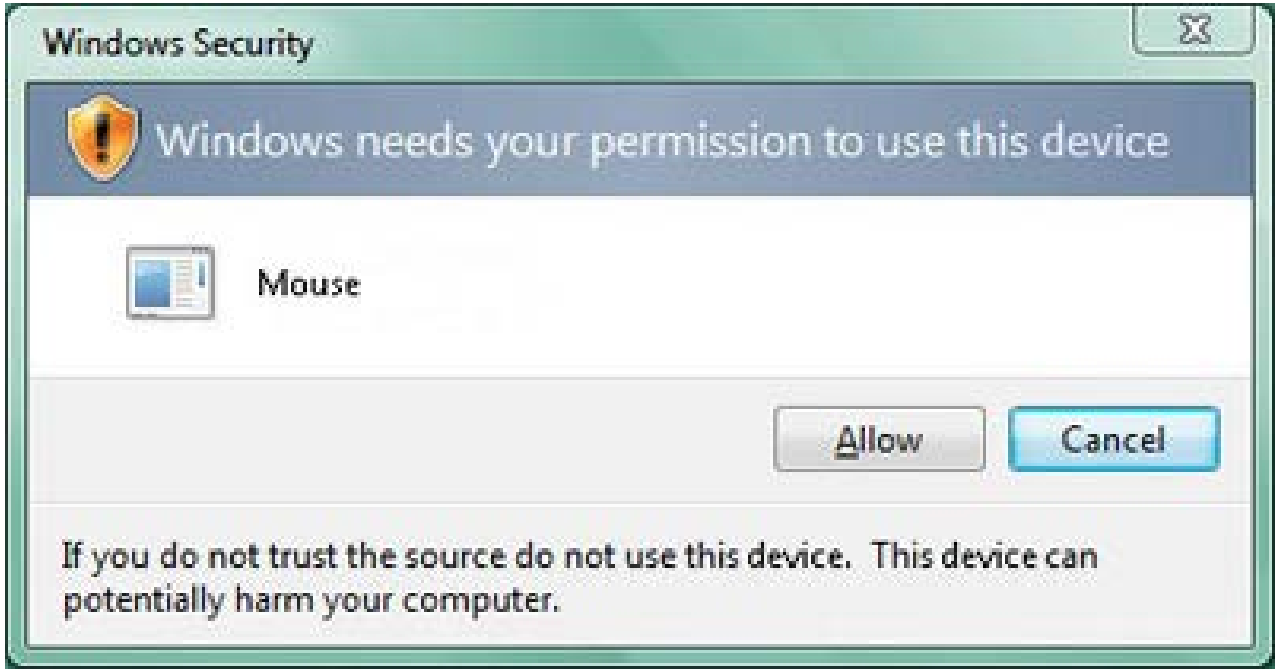
Logistics Questions?

What is security?

- Computer Security is, given an attacker's model, the technique to control who may use or modify the computer or the information contained in it.

Security Theatre

- Bruce Schneier — measures designed to produce a feeling of security rather than the reality



Security vs. Reality

- Don’t protect $1B with encryption that can be broken for $1M.
- Don’t spend $10M to protect $1M.
- Interesting map
  - https://cybermap.kaspersky.com/

Example - OS Security Design

- CVE-2015-7001 : Razvan Deaconescu and Mihai Chiroiu of University POLITEHNICA of Bucharest; Luke Deshotels and William Enck of North Carolina State University; Lucas Vincenzo Davi and Ahmad-Reza Sadeghi of TU Darmstadt
- Description: An issue existed in the sandbox's handling of hard links. This issue was addressed through improved hardening of the app sandbox.

Example - Application Security

- CVE-2014-0160 (Heartbleed): (Riku, Antti and Matti) at Codenomicon and Neel Mehta of Google Security
- Description: Bug is in the OpenSSL's implementation of the TLS/DTLS (transport layer security protocols) heartbeat extension (RFC6520). When it is exploited it leads to the leak of memory contents from the server to the client and from the client to the server.

Example - Hardware Security

- Chip and Skim: cloning EMV cards with the pre-play attack *Mike Bond, **Omar Choudary,** Steven J. Murdoch, Sergei Skorobogatov, and Ross Anderson*
- Description: “We have discovered that some EMV implementers have merely used counters, timestamps or home-grown algorithms to supply this number.[…] Card cloning is the very type of fraud that EMV was supposed to prevent.”

Example - Access Control

- Description: “In short, the very four digits that Amazon considers unimportant enough to display in the clear on the web are precisely the same ones that Apple considers secure enough to perform identity verification.”

Example - Network Security

- DDOS of 300 Gbps Cyberbunker vs Spamhaus
- Description: “DNS amplification attacks “

Example - Web Security

- https://www.owasp.org/index.php/Main_Page

Example - Cryptography





Example - Cryptography

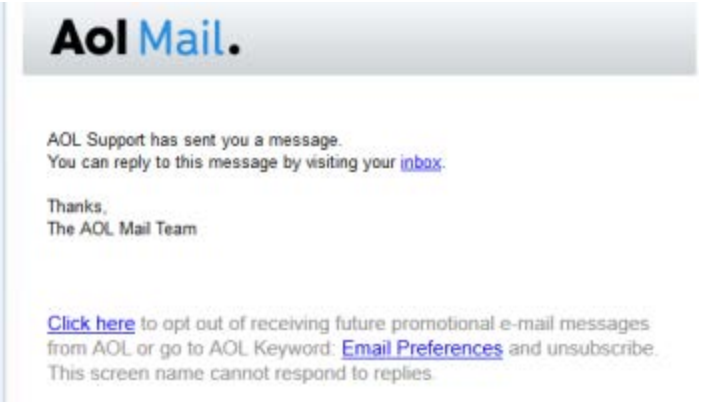
- MD5 – Developed in 1992
- Description (2004): “How to Break MD5 and Other Hash Functions” Xiaoyun Wang and Hongbo Yu
- Usage (2008): http://www.win.tue.nl/hashclash/rogue-ca/

Example - Usable Security

- Description “Usability is one of the most important and yet hardest design problems in many secure systems.” *by Ross Anderson*

- Technology writer David Pogue calculated we spend 17 man-years every day on CAPTCHAs (Scientific American, March 2012)

- Phishing
  - 1996



- “Given a choice between dancing pigs and security, users will pick dancing pigs every time.” *by Edward Felten and Gary McGraw*

Dancing pigs!



Romanian Legislation (not translated)

- Introducerea, modificarea sau ştergerea de date informatice, restricţionarea accesului la aceste date ori împiedicarea în orice mod a funcţionării unui sistem informatics [...] se pedepseşte cu închisoarea de la 2 la 7 ani.
- (1) Accesul, fără drept, la un sistem informatic se pedepseşte cu închisoare de la 3 luni la 3 ani sau cu amendă.
- (2) Fapta prevăzută în alin. (1), săvârşită în scopul obţinerii de date informatice, se pedepseşte cu închisoarea de la 6 luni la 5 ani.
- (3) Dacă fapta prevăzută în alin. (1) a fost săvârşită cu privire la un sistem informatic la care [...] accesul este restricţionat sau interzis pentru anumite categorii de utilizatori, pedeapsa este închisoarea de la 2 la 7 ani.

Security Theatre

Security Basics

- Confidentiality
  - Prevent the disclosure of sensitive information from unauthorized people, resources, and processes
- Integrity
  - The protection of system information or processes from intentional or accidental modification
- Availability
  - The assurance that systems and data are accessible by authorized users when needed

Confidentiality

- Data protection/personal data privacy
  - fair collection and use of personal data, in Europe a set of legal requirements
- Anonymity/untraceability
  - ability to use a resource without disclosing identity/location
- Unlinkability
  - ability to use a resource multiple times without others being able to link these uses together
  - Bad examples: HTTP “cookies”
- Pseudonymity
  - anonymity with accountability for actions.
- Unobservability
  - ability to use a resource without revealing this activity to third parties
- Copy protection, information flow control
  - ability to control the use and flow of information

Integrity and Availability

- Rollback
  - ability to return to a well-defined valid earlier state (backup, revision control, undo)
- Authenticity – verification of the claimed identity of a communication partner
- Non-repudiation – origin and/or reception of message cannot be denied in front of third party
- Audit – monitoring and recording of user-initiated events to detect and deter security violations
- Intrusion detection – automatically notifying unusual events

What is there to secure?



- Stored data
  - Personal information (employees, customers, users, etc)
  - Copyrighted software
  - Securing data must also ensure persistence
    - Data must not be lost due to attacks or lack of skill
- Transactions
  - Protect information from being tampered with
  - Make sure that the sender is who he/she claims to be
  - Make sure the receiver is the one intended
  - Data is often sent across public (insecure) networks – it can easily be intercepted
- Secure access
  - Access to computers / networks / certain privileges

Security Administration

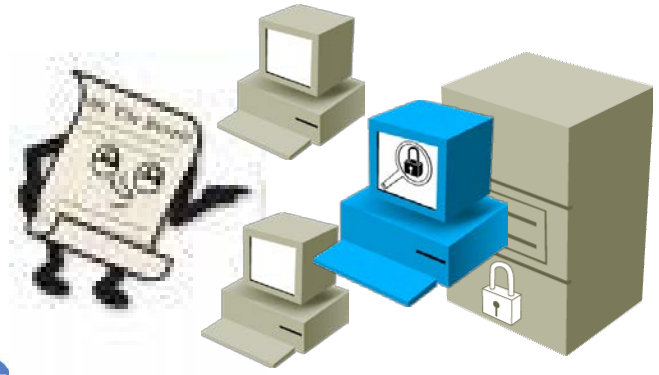
- Policies
- Standards
- Guidelines
- Procedures
- Baselines



© Mihai Chiroiu

What Is a Security Policy?

- A document that states how an organization plans to protect its tangible and intangible information assets
  - Management instructions indicating a course of action, a guiding principle, or appropriate procedure
  - High-level statements that provide guidance to workers who must make present and future decisions
  - Generalized requirements that must be written down and communicated to others



Documents Supporting Policies

- Standards – dictate specific minimum requirements in our policies
- Guidelines – suggest the best way to accomplish certain tasks
- Procedures – provide a method by which a policy is accomplished (the instructions)



Security and complexity

- Downside: Complexity brings vulnerability
  - How secure is a 1000-computer network with >1000 users and 200 different applications?
  - How secure is a simple button?

- Still, we DO need complexity to accomplish our tasks

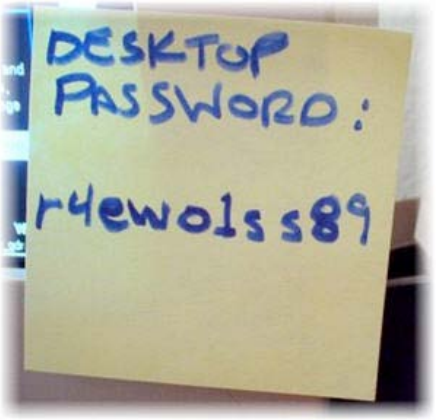


Least privilege

- Complex systems are more difficult to secure.
- The more application deployed, the more possible vulnerabilities.



Security and humans

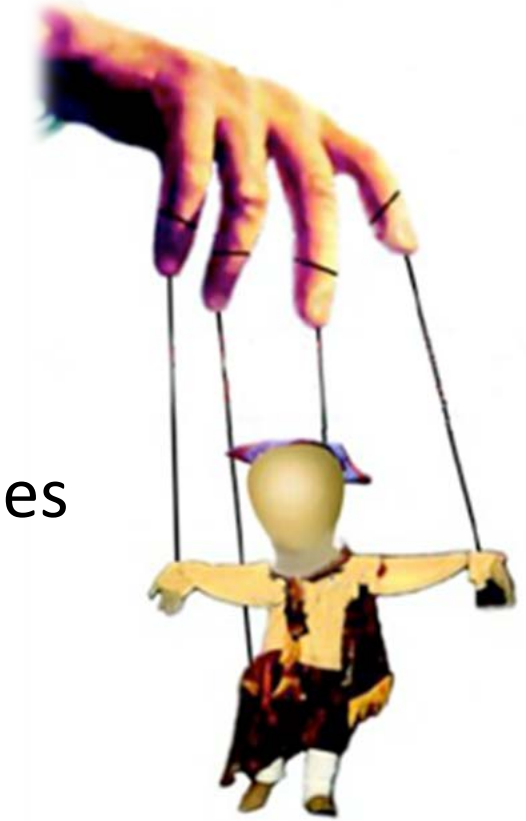


- Security policies must be in place

  …and must be followed.

- Regardless of how strong (and expensive) your secure deployment is:
  - Humans can still write their passwords on post-it notes
  - Humans can still give their passwords to anyone they trust
  - Humans can still open tempting attachments…

Social engineering



- Non-technical intrusion
- Involves tricking people to break security policies
  - Manipulation
- Relies on false confidence
  - Everyone trusts someone
  - Authority is usually trusted by default
  - Non-technical people don’t want to admit their lack of expertise
    - They ask fewer questions.
  - Most people are eager to help.
    - When the attacker poses as a fellow employee in need.

Social engineering

- People are not aware of the value of the information they posess.
- Vanity, authority, eavesdropping – they all work.
- When successful, social engineering bypasses ANY kind of security.



References

1. http://www.informationisbeautiful.net/visualizations/worlds-biggest-data-breaches-hacks/
2. http://www.phishing.org/history-of-phishing/
3. https://www.owasp.org/images/2/25/OWASP_angela_sasse_appsec_eu_aug2013.pdf
4. http://www.criminalitate.info/2014/02/noul-cod-penal-2014-infractiuni-informatice-criminalitate-informatica.html
5. http://www.wired.com/2012/08/apple-amazon-mat-honan-hacking/

References

6. http://arstechnica.com/security/2013/03/spamhaus-ddos-grows-to-internet-threatening-size/
7. https://www.us-cert.gov/ncas/alerts/TA13-088A

Introduction to Cryptographic Systems

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

Vocabulary

- In cryptography, **cyphertext** is the result of **encryption** performed on **plaintext** using an algorithm, called a **cipher**.

- **Decryption** is the process of turning ciphertext into readable plaintext.

- This lecture is not about **steganography**: the practice of concealing a file/message/image/video within another file/message/image/video.

1500 BCE - clay tablets from Mesopotamia



Fast forward on crypto history

- Caesar cipher (100 BCE – 44 BCE)
- Most of Caesar's enemies would have been illiterate => secure
- The shift cipher
- Vigenère cipher (1553 CE) improves on Caesar's cipher
- 1st & 2nd WW led to cipher machines, both for encryption, decryption and for cracking



Modern cryptography

- Claude Shannon (1949)
  - What does an ideal cipher look like?

- **One-time Pad** protects against **infinitely** powerful adversaries
  - Bad news: If you want to encrypt N bits of data, you need a N-bit secret key

From military to business





Encryption Schemes

- **Public Algorithms** – All the details of the algorithm are in the public domain, known to everyone.
  - Kerckhoffs' principle (Dutch cryptographer): *A cryptosystem should be secure even if everything about the system, except the key, is public knowledge.*
  - Reformulated as Shannon's maxim (American mathematician): "the enemy knows the system", i.e., "*one ought to design systems under the assumption that the enemy will immediately gain full familiarity with them*".
- **Proprietary algorithms** – The details of the algorithm are only known by the system designers and users.
  - security through obscurity.

DES (and 3DES)

- Developed by International Business Machines (IBM) as LUCIFER and modified by the National Security Agency (NSA).
  - LUCIFER used a key size of 128 bits however this was reduced to 56 bits for DES. ☺
- Adopted in 1977 as the Data Encryption Standard - DES

Problem description

- Can two parties agree on a shared secret without private communication? [Ralph Merkle]

Diffie - Hellman

- Take Merkle's idea and improves it such that the attacker requires exponential computations

RSA

- Noticing the inability of the Diffie-Hellman Key Exchange to transmit a secret message, Ron Rivest, Adi Shamir, and Leonard Adleman developed a system similar to the Diffie-Hellman protocol except that a message could be embedded and transmitted.

Elliptic Curve Cryptography

- ECC is an approach to encryption that utilizes the complex nature of elliptic curves in finite fields. ECC typically uses the same types of algorithms as that of Diffie-Hellman Key Exchange and RSA Encryption. The difference is that the numbers used are chosen from a finite field defined within an elliptic curve expression.

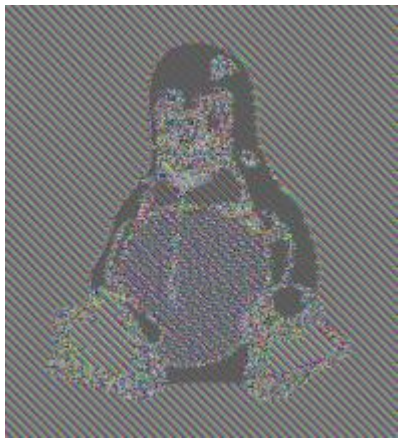
AES

- In January 1997, NIST announced a competition for the successor to DES.
- NIST selected the winner, the Rijndael (pronounced "Rhine doll") algorithm of Belgian cryptographers Joan Daemen and Vincent Rijmen in October 2000.
- AES was approved for use with Secret and Top Secret classified information of the U.S. government in 2003.

Block ciphers mode of operation

- *Partition into n-bit blocks*
- Electronic Codebook (ECB)
  - Each block encrypted independently
  - Identical plaintexts encrypted similarly
  - No chaining, no error propagation
  - Does not hide data patterns, unsuitable for long
  - messages





Block ciphers mode of operation

- Cipher-Block Chaining (CBC)
- Allows random access to ciphertext
- Decryption is parallelizable
- Identical messages: changing IV or the first plaintext block results in different ciphertext
- Chaining: Ciphertext block cj depends on xj and all preceding plaintext blocks (dependency contained in cj-1)
- Error propagation: Single bit error on cj may flip the corresponding bit on xj+1, but changes xj significantly.
- IV need not be secret, but its integrity should be protected

Block ciphers mode of operation

- Cipher Feedback (CFB),
- Allows random access to ciphertext
- Decryption is parallelizable
- Identical messages: as in CBC
- Chaining: Similar to CBC
- Error propagation: Single bit error on cj may flip the corresponding bit on xj
- , but changes xj+1 significantly.
- IV need not be secret (XORed with x1)

Block ciphers mode of operation

- Output Feedback (OFB)
- Preprocessing possible (keep enc/decrypting previous output block)
- No random access, not parallelizable
- Identical messages: same as CBC
- No chaining dependencies
- Error propagation: Single bit error on cj may only affect the corresponding bit of xj
- IV need not be secret, but should be changed if a previously used key is to be used again

Block ciphers mode of operation

- Counter (CTR)
- Preprocessing possible (inc/decrement and enc/decrypt counter)
- Allows random access
- Both encryption & decryption are parallelizable
- Identical messages: changing nonce results in different ciphertext
- No chaining dependencies
- No error propagation
- Nonce should be random, and should be changed if a previously used key is to be used again

Which Mode for What Task?

- General file or packet encryption: CBC.
  - Input must be padded to multiple of cipher block size
- Risk of byte or bit deletion: CFB
- Bit stream; noisy line and error propagation is undesirable: OFB
- Very high-speed data: CTR

- *Integrity check is required*

HASH

Attacks On Cryptosystems

Passive Attacks

- No communication with victim
- Stealing of private data without the know of the victim

Active Attacks

- Involves changing data with victim
- Unauthorized data access in order to modify/delete/alter it

Types of attacks

- **Ciphertext Only Attacks (COA)** – The attacker has access to a set of ciphertext(s), and not to the corresponding plaintext. Successful when the corresponding plaintext can be determined from a given set of ciphertext.
- **Known Plaintext Attack (KPA)** – The attacker knows the plaintext for some parts of the ciphertext. The task is to decrypt the rest of the ciphertext using this information. Typically, this may be done by determining the key.
- **Chosen Plaintext Attack (CPA)** – The attacker has the text of his choice encrypted. This simplifies his task of determining the encryption key.
- **A chosen-ciphertext attack (CCA) –** The attacker can gather information by obtaining the decryptions of chosen ciphertexts. The adversary can attempt to recover the key used for decryption.

Types of attacks

- **Dictionary Attack** – The attacker builds a dictionary of ciphertexts and corresponding plaintexts that he has learnt over a period of time. In future, when an attacker gets the ciphertext, he refers the dictionary to find the corresponding plaintext.
- **Brute Force Attack (BFA)** – In this method, the attacker tries to determine the key by attempting all possible keys.
- **Man in Middle Attack (MIM)** – The targets of this attack are mostly public key cryptosystems where key exchange is involved before communication takes place.
- **Side Channel Attack (SCA)** – This type of attack is launched to exploit the weakness in physical implementation of the cryptosystem.
- **Timing/Power Analysis Attacks** – They exploit the fact that different computations take different times to compute on processor.

Practicality of Attacks

- Highly academic

- Unrealistic assumptions
  - In chosen-ciphertext attack, the attacker requires an impractical number of deliberately chosen plaintext-ciphertext pairs.

Post-quantum cryptography

- Equation 1 shows the time it takes to run the fastest known algorithm (GNFS) to compute a prime factorization on a binary formatted processor. Equation 2 shows the algorithm discovered by Peter Shor that computes a prime factorization on a quantum computer. In both cases, "b" is the number of bits in the number.

$$e^{\left[\left(\frac{64}{9}*b\right)^{\frac{1}{3}}(\log b)^{\frac{2}{3}}\right]}$$

Equation 1. GNFS algorithm time.

$$b^3$$

Equation 2. Shor's algorithm time.

Resources

- https://en.wikipedia.org/wiki/History_of_cryptography
- https://www.tutorialspoint.com/cryptography/attacks_on_cryptosystems.htm
- https://en.wikipedia.org/wiki/Caesar_cipher
- http://www.umsl.edu/~siegelj/information_theory/projects/des.netau.net/des%20history.html
- http://www.utdallas.edu/~muratk/courses/crypto07_files/modes.pdf

Resources

- http://www.crypto-it.net/eng/theory/modes-of-block-ciphers.html
- https://www.sans.org/reading-room/whitepapers/vpns/history-encryption-730
- http://www.eng.utah.edu/~nmcdonal/Tutorials/EncryptionResearchReview.pdf

Hardware Security

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

This lecture

- Is **not** about:
  - Hardware Trojan
  - Side-channel attacks on hardware
  - Physically Unclonable Function
  - Other: e.g., Cold boot attack

- Is about:
  - Root-of-trust
  - Hardware- assisted computer security: TPM, ArmTrustZone, IntelSGX

Trustworthy Computing

- Goal: Protect data from misuse
- Approach: Turn a portion of a platform into a trustworthy environment
- TCB is not sufficient

Security Properties [1]

- Isolated Execution
  - Inside a Trusted Execution Environment (TEE)
- Secure Storage
  - Integrity, confidentiality
- Attestation (remote and local)
  - Data given only to the trusted machine
- Secure Provisioning
  - Channel for sending data
- Trusted Path
  - Communication channel for peripherals (Secure I/O)

Trusted Platform Module (TPM)

Why?

- “For years Bill Gates has dreamed of finding a way to make the Chinese pay for software, TC looks like being the answer to his prayer.” by Ross Anderson

TPM history [2]

- The Trusted Computing Platform Alliance (TCPA) The Trusted Computing Platform Alliance (TCPA)
  - Established by the 5 founders in 1999: Intel, AMD, IBM, HP and MSFT
  - TPM v1
- The Trusted Computing Group (TCG)
  - Established in March 2003 as continuation of TCPA
  - TPM v2

TPM Capabilities [3]

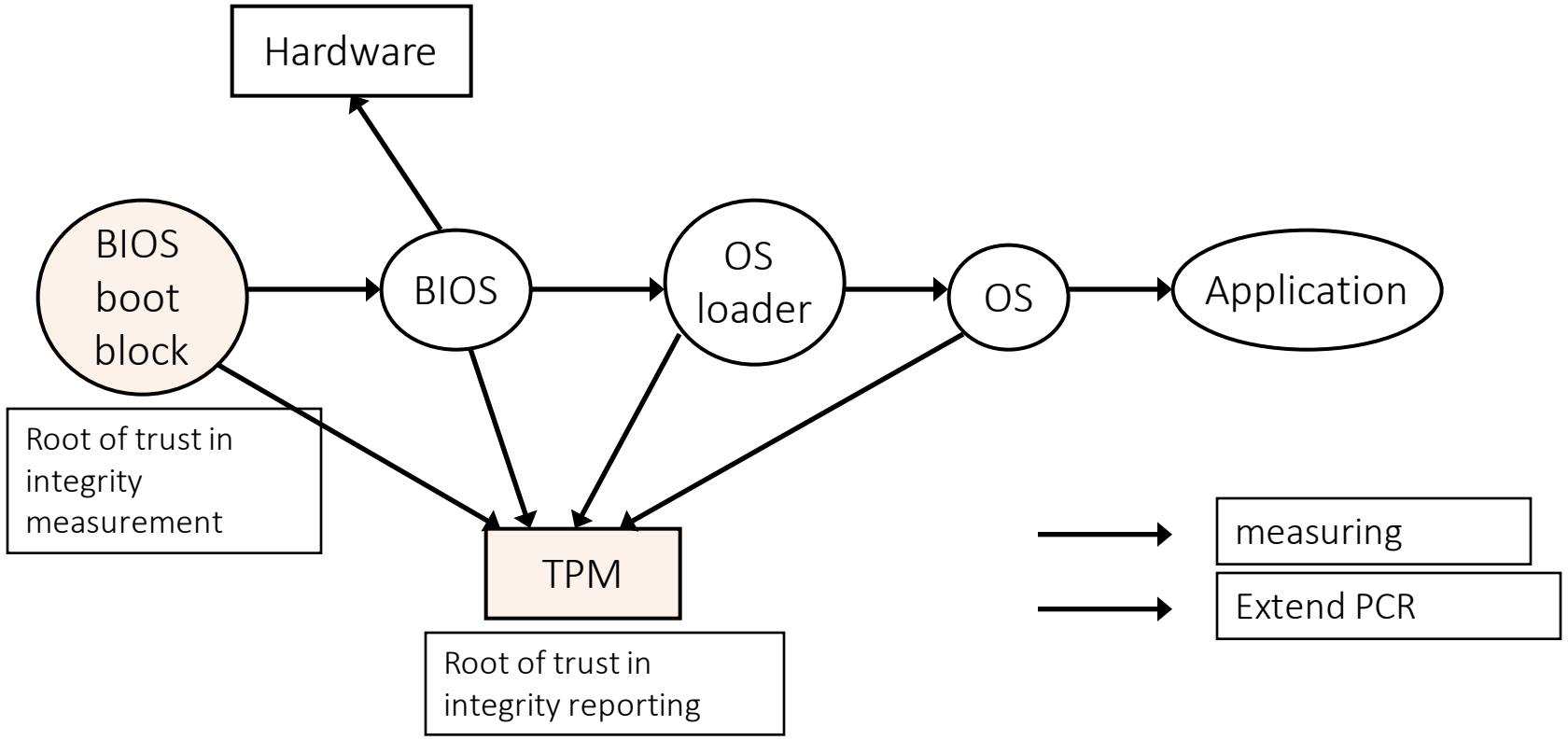
- Provides three root of trusts:
  - Root of trust for measurement (RTM) a trusted implementation of a hash algorithm
  - Root of trust for storage (RTS) a trusted implementation for one or more secret keys —the storage root key (SRK)
  - Root of trust for reporting (RTR) a trusted implementation for a secret key representing a unique platform identity, the endorsement key, (EK).
    - Signed by the platform vendor.

TPM Capabilities

- ‘Platform Configuration Registers’ (PCR)
  - Can be read
  - Can only be extended, not writable
- Migratable vs non-migratable keys
  - EK and SRK never leave TPM

Trusted/Secure Boot

- After boot, PCRs contain hash chain of booted software



Secure Storage

- Step 1: TPM_TakeOwnership(OwnerPassword, …)
  - The SRK is created and can be deleted

- Binding/Unbind vs Sealing/Unsealing
  - Sealing is an extension to binding.
  - Contrary to binding, only non-migratable storage keys can be used to seal data.
  - Consequently, the encrypted data is always bound to a specific platform.

Attestation (remote and local)

- Why is remote attestation different from local attestation?
  - Answer: Because one's computation capabilities (i.e., local one has do to the cryptographic part on paper)

Attestation (remote and local)

- Step 1: Create Attestation Identity Key (AIK)
- Step 2: Sign PCR values (after boot)
- Step 3: Validate signed PCRs
  - How to do that for local attestation?

- Problems?
  - It only validates the loaded code, not the running one
  - Private attestation (cannot tell what machine it came from)
    - Privacy CA

Secure Provisioning

- Can be done using sealing
- Similar to local attestation

Trusted Path

- No real support from TPM
- Using Dynamic Root of Trust

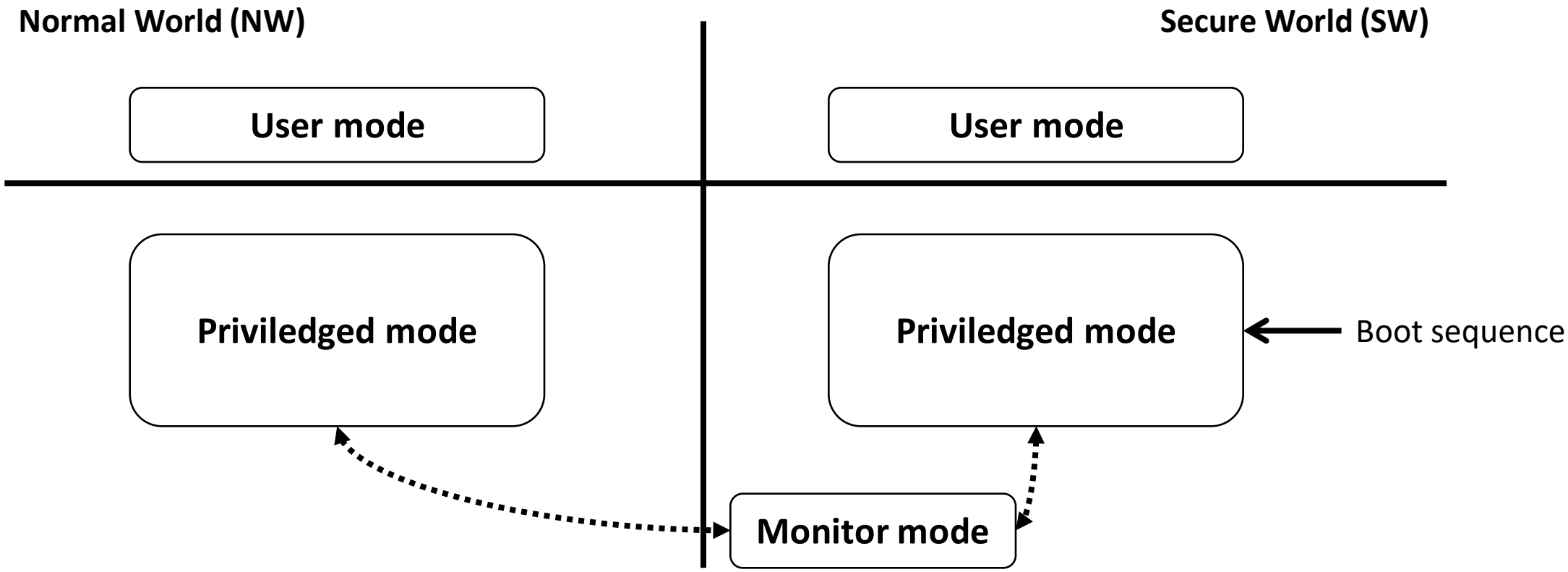
TPM Controversy

- Could be used quite coercively
  - E.g., web pages only readable by browser X
  - Documents only usable with word processor Y

- Vendor lock-in

ARM TrustZone

TrustZone Capabilities



- The entry to monitor execute the Secure Monitor Call (SMC)

Isolated Execution

- By design
- Two virtual Memory Management Units (one for each state)
  - The secure world can access the normal world data
- Can also implement secure boot

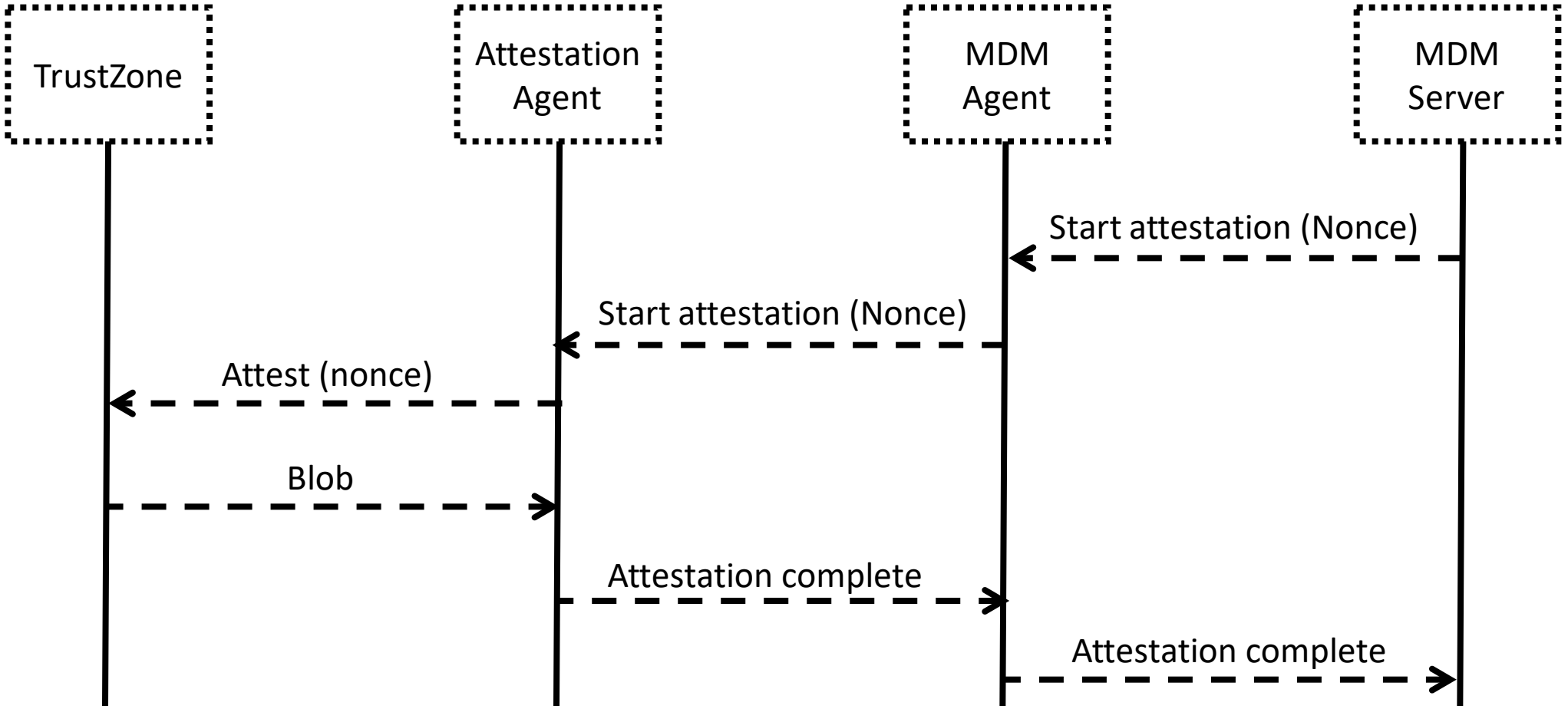
Attestation – Case Study KNOX

- Samsung's TrustZone-based Integrity Measurement Architecture (TIMA)
- TIMA Periodic Kernel Measurement (PKM)
- TIMA Real-time Kernel Protection (RKP)
  - intercepts critical kernel events, which are then inspected in TrustZone

Attestation – Case Study KNOX

- Attestation Blobs are signed by TIMA
- Unique device public/private key pair (starting with the Note 3)
- Certificate for device key is signed with Samsung root key
- TIMA generates an attestation public/private key pair, and signs a certificate for the attestation key using the device private key
- Attestation private key is used to sign data inside attestation blob

Attestation – Case Study KNOX

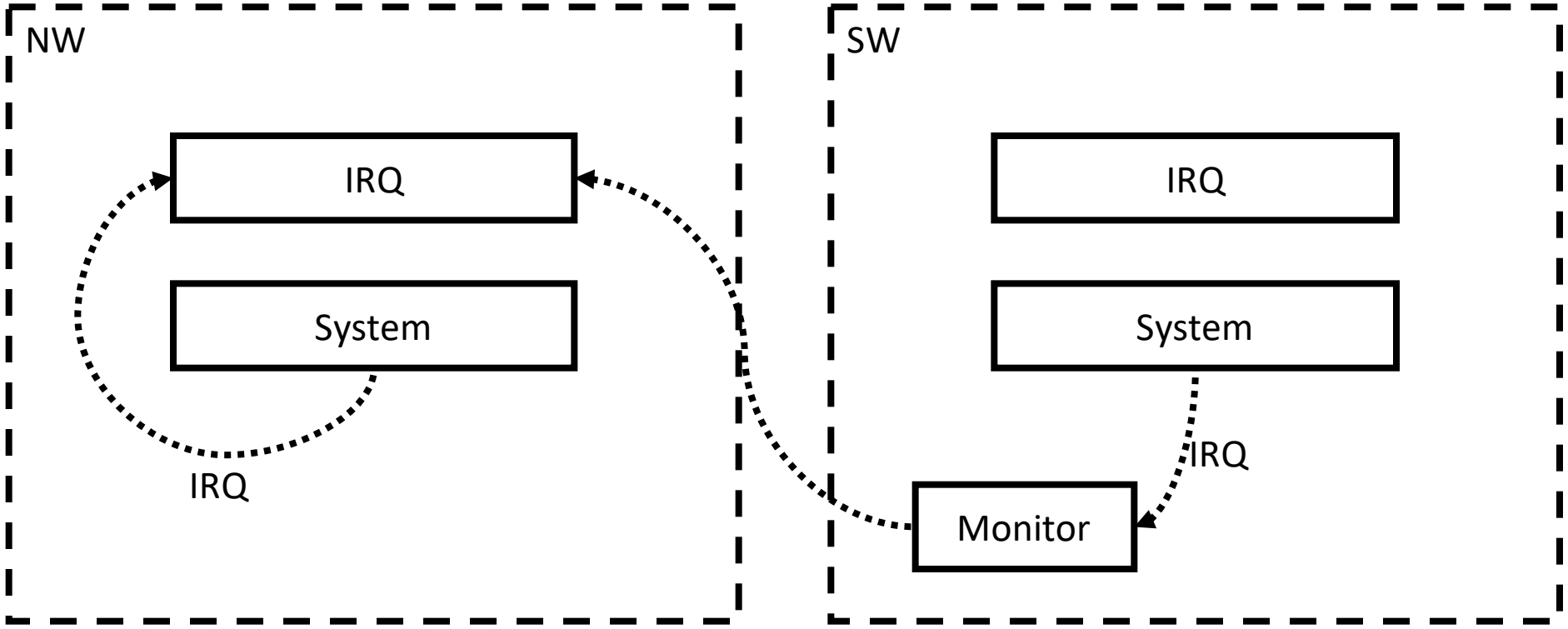


Other capabilities

- Secure Storage and Provisioning can be implemented using TIMA

Trusted Path

- The ability to trap IRQ and FIQ directly to the monitor



TrustZone Discussion

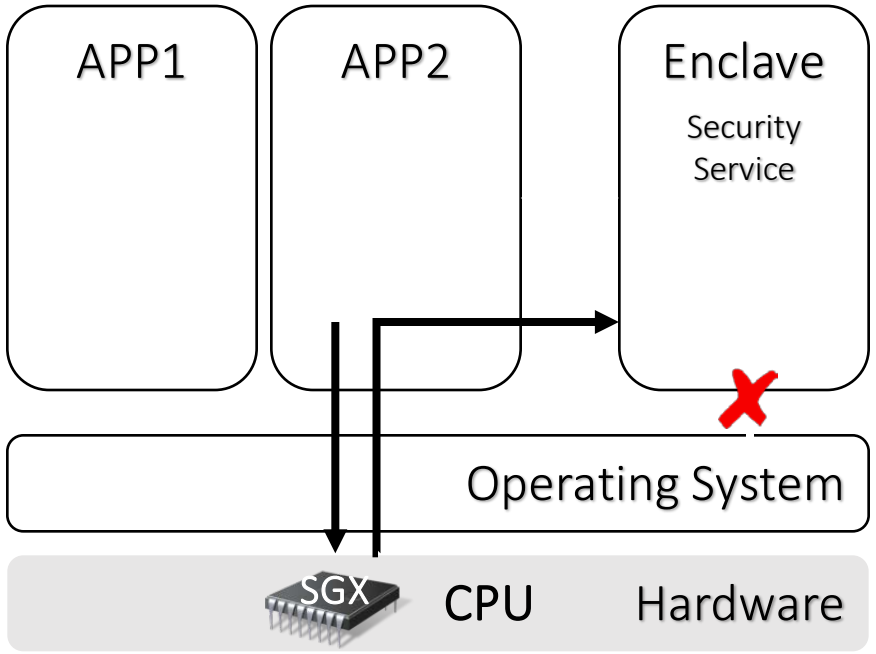
- Vulnerabilities in the TEEOS (CVE-2015-4421)
- Closed system for third application development
- Only one compartment for TCB
- No Virtualization support

Intel Software Guard Extensions (SGX)

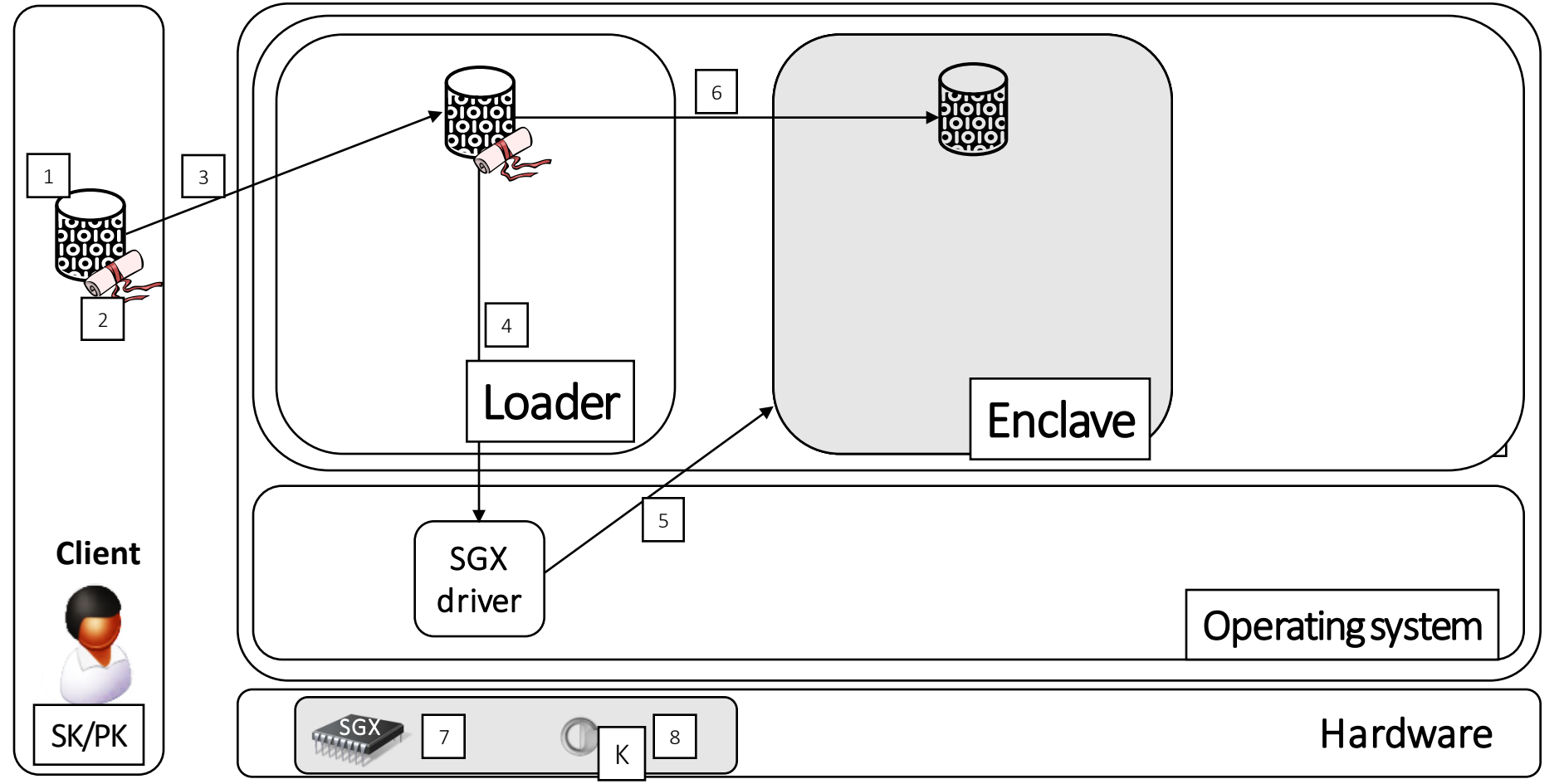
SGX Capabilities

- Security critical code isolated in enclave
- Only CPU is trusted
  - Transparent memory encryption
- Enclaves cannot harm the system
  - Only unprivileged code (CPU ring3)
  - Memory protection
- Designed for Multi-Core systems
  - Multi-threaded execution of enclaves
  - Parallel execution of enclaves and untrusted code
  - Enclaves are interruptible

Isolated Execution



Trusted | Untrusted



1. Create App
2. Create app certificate (includes HASH(App) and Client PK)
3. Upload App to Loader
4. Create enclave
5. Allocate enclave pages
6. Load & Measure App
7. Validate certificate and enclave integrity
8. Generate enclave K key
9. Protect enclave

Trusted | Untrusted

© Mihai Chiroiu

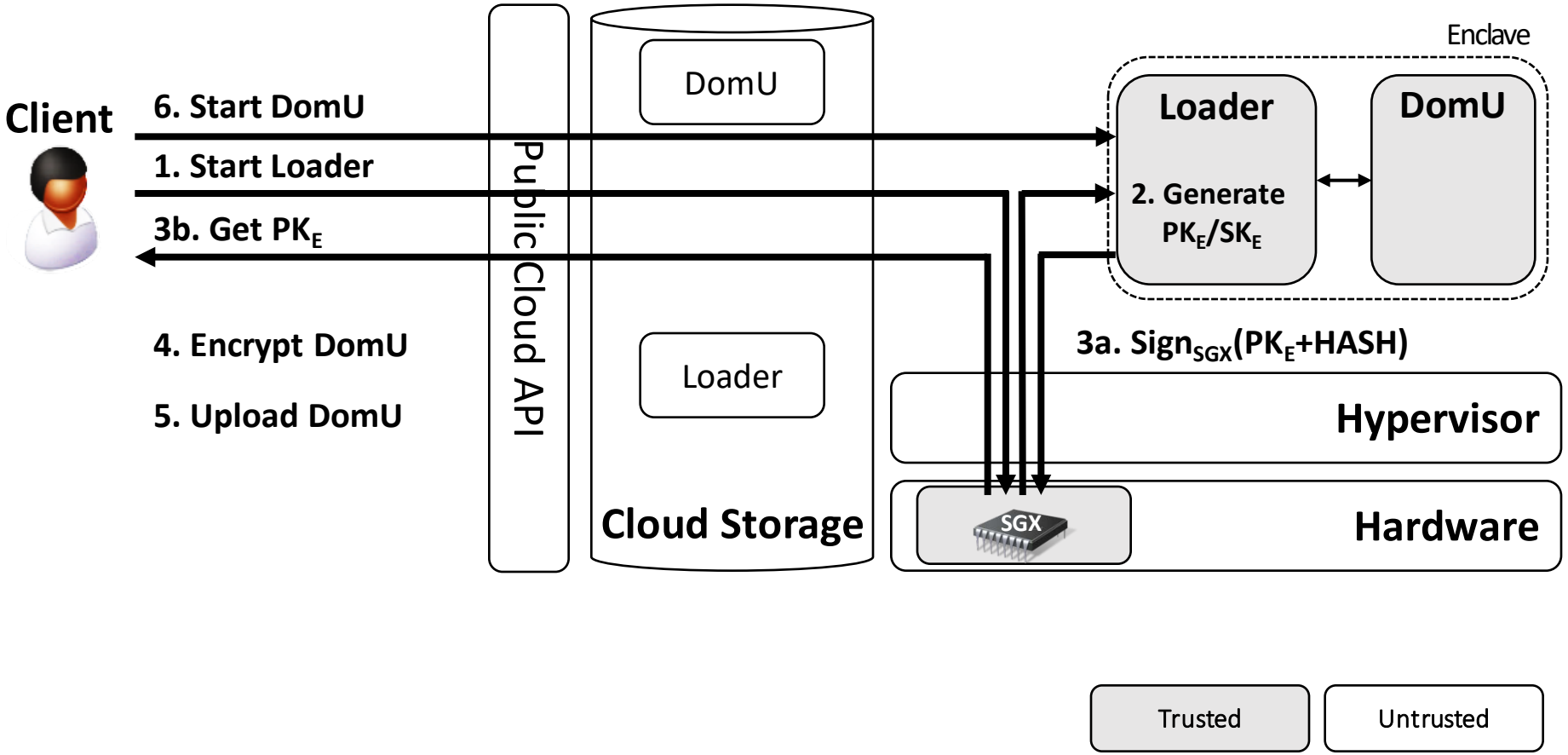
SYSTEMS LABORATORY

Computer Science & Engineering Department

Attestation (remote and local)

- Yes, built-in with SGX
- An enclave can request a HW-signed REPORT
- In local attestation, one enclave can attest its TCB to another one

Secure Provisioning – Use Case VMM



Secure Storage

- Based on the Secure Provisioning of Key

Trusted Path

- Open research question

Open Challenges for SGX

- Malware in enclave
  - Check against blacklist before loading
  - White listed code can be exploited
    - Runtime attacks possible
  - Detect malicious behavior → block enclave
    - Enclave by itself cannot do much harm (memory protection)

Open Challenges for SGX

- VM Migration in the cloud
  - How can it be done transparent to the VM (the VMs OS)

- Side Channel attacks
  - Enclaves are interruptible
  - Caches are not flushed on switching between SGX and non-SGX mode
    - Data oblivious algorithms required [Kreuter et al., USENIX'13]

Comparation

	Isolated Execution	Secure Storage	Remote Attestation	Secure provisioning	Trusted Path
TPM	No	Yes (limited)	Yes	Yes	No
TrustZone	Yes	Yes	Yes	Yes	Yes
SGX	Yes	Yes	Yes	Yes	Probably

Hardware Security Modules (HSM)

HSM functionalities

- A piece of hardware and associated software/firmware that usually attaches to the inside of a PC or server and provides at least the minimum of cryptographic functions.

- Strong random number generation
- A secure time source
- Tamper-resistance

Security Keys [6]

- HSM are exposed to direct access attacks
- Already available in Chrome



References

1. http://repository.cmu.edu/cgi/viewcontent.cgi?article=1096&context=cylab
2. IBM.Press.A.Practical.Guide.to.Trusted.Computing.Jan.2008
3. https://www.cs.ox.ac.uk/files/1873/RR-08-11.PDF
4. http://asokan.org/asokan/Padova2014/tutorial-mobileplatsec.pdf
5. http://resources.infosecinstitute.com/uefi-and-tpm/
6. http://fc16.ifca.ai/preproceedings/25_Lang.pdf

References

7. http://www.cse.unsw.edu.au/~cs9242/15/exam/paper2.pdf
8. http://www.samsung.com/ro/business-images/insights/2015/An_Overview_of_the_Samsung_KNOX_Platform_V1.11-0-0.pdf

Access Control

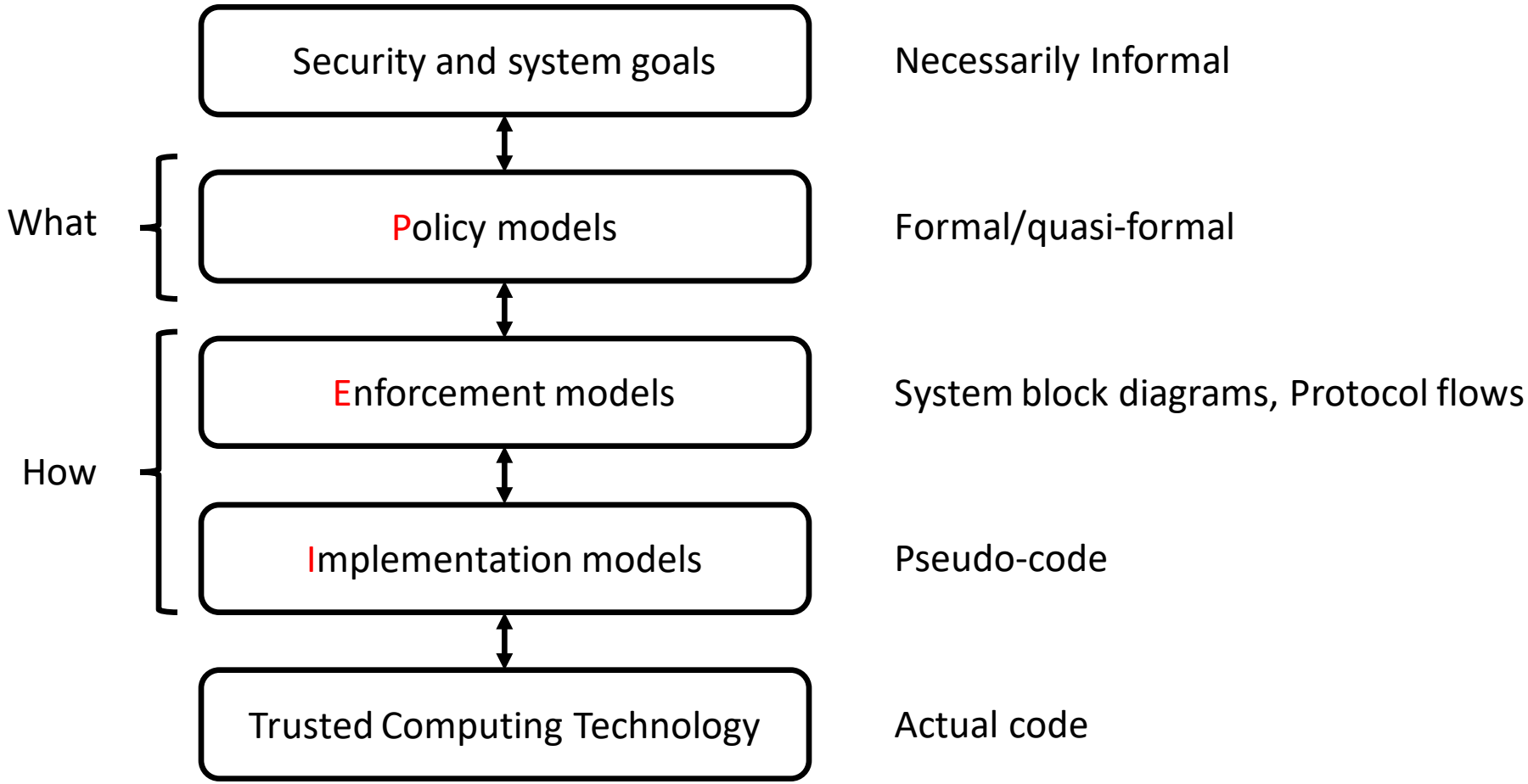
Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

Examples of Access Control

- Social Networks: Access to personal information.
- Web Browsers: Javascript - same origin policy.
- Operating Systems: One user cannot arbitrarily access/kill another user's files/processes.
- Memory Protection: Code in one region (for example, in Ring 3), cannot access the data in another more privileged region (e.g. Ring 0).
- Firewalls: If a packet matches with certain conditions, it will be dropped.

PEI Model [Sandhu, 2006]



© Mihai Chiroiu

SYSTEMS LABORATORY

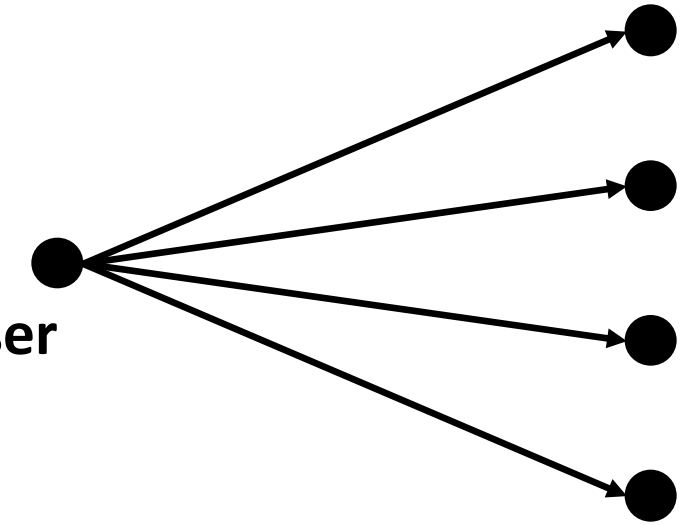
Computer Science & Engineering Department

Vocabulary

- Basic abstractions:
  - Subjects
  - Objects
  - Rights

- A **subject** is an entity who wishes to access a certain **object**, which is some kind of resource, e.g., a file or directory. The different modes of access (e.g., reading, writing) are called **permissions**.

Vocabulary – Users and Principals

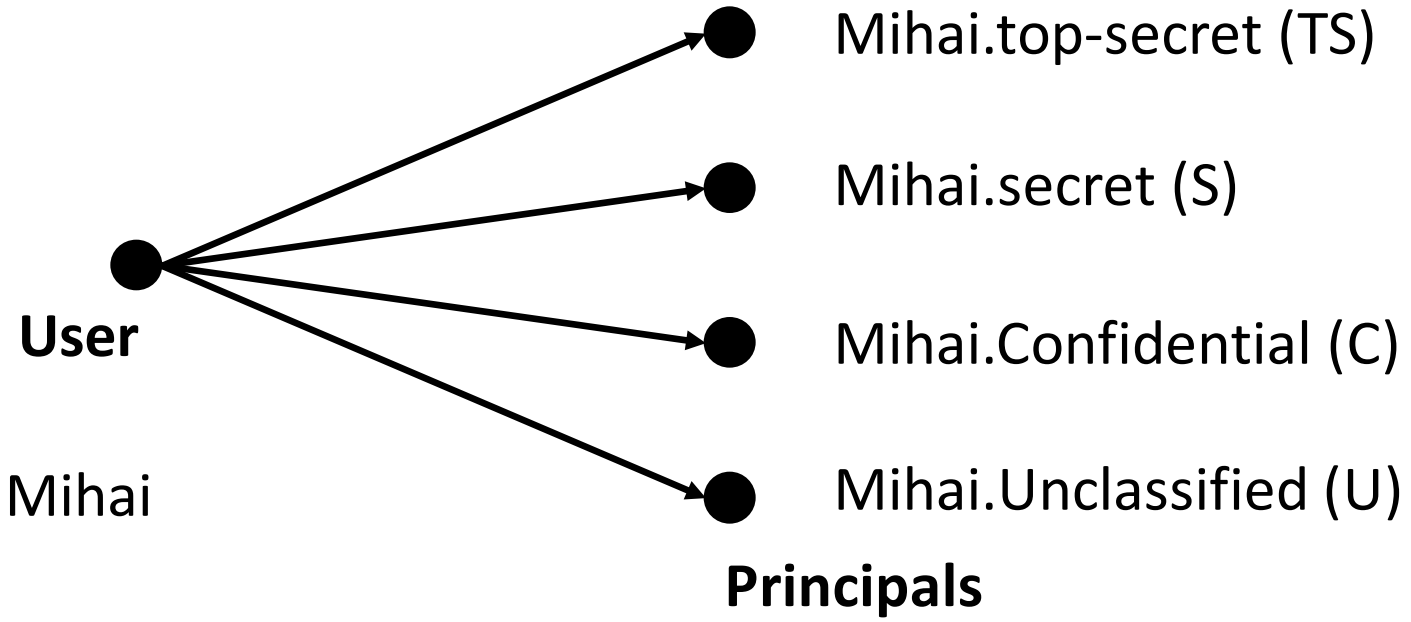


Real World **User**

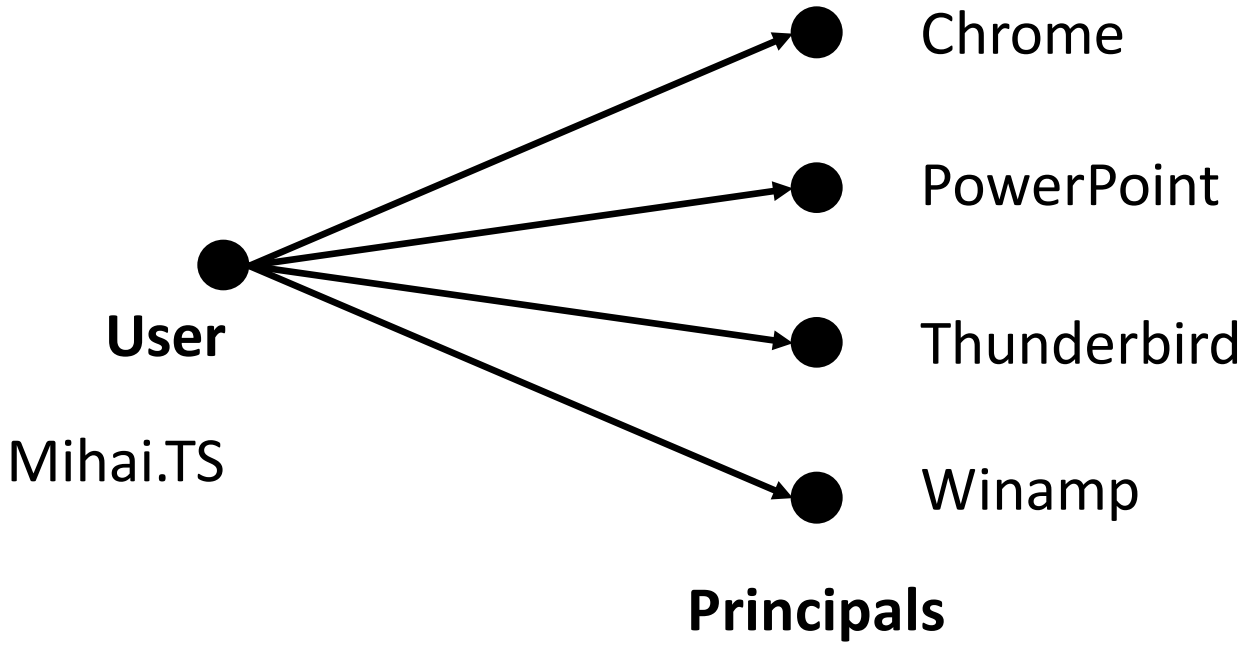
Principals - Unit of Access Control and Authorization

- A Principal is an User authenticated in a context

Vocabulary – Users and Principals



Vocabulary – Principals and subjects



- A subject is a program executing on behalf of a principal

Vocabulary

- The relation between Users and Principals is One-To-Many
  - Allows accountability of user's actions
  - Don't share your passwords!!!

- For simplicity, a principal and subject can be treated as identical concepts.

Vocabulary - Objects

- An object is anything on which a subject can perform operations (mediated by rights)
- Usually objects are passive, for example:
  - File
  - Directory (or Folder)
  - Memory segment
- But, subjects can also be objects, with operations
  - kill
  - suspend
  - resume

Access control models

Access control enforcement

- **Discretionary access controls (DAC)** – the access of objects (or subjects) can be propagated from one subject to another. Possession of an access right by a subject is sufficient to allow access to the object.
- **Mandatory access controls (MAC)** – the access of subjects to objects is based on a system-wide policies (based on security labels) that can be changed only by the administrator.
- **Role-Based Access Control (RBAC)** – can be configured as both MAC or DAC, access to objects is based on roles.

Discretionary access controls

DAC

- No precise definition.
- Basically, DAC allows access rights to be propagated at subject's discretion
  - often has the notion of owner of an object
  - used in UNIX, Windows, etc.

DAC Implementation

- Let *S* be the set of all subjects, *O* the set of all objects, and P the set of all permissions. The description of access control can be given by a set $A \subseteq S \times O \times P$.
- When new permissions are added, triplets are added to A; when they are removed (revoked), triplets are deleted.

Access control – Representation

- An access control matrix is a matrix (Ms,o) whose rows are subjects and columns are objects. Element (Ms,o) ⊆ P is the set of permissions that subject **S** is authorized for object o.

Objects (and Subjects) →

Subjects ↓

	A	B	C	D
U1				
U2		rw		kill
U3			parent	
U4		r		

Access Control Lists (ACL)

- An access control list is a set {Ao | o ∈ O}, one element for each **object**. The elements of the list are the pairs (s, p) of **subjects** s who have **permission** p to that object.

B
U2: rw
U4: r

C
U3: parent

D
U2: Kill

Capabilities

- Storing capabilities means giving to each subject tokens which give them access to the permissions they are entitled.

U1	
U2	B/rw, D/kill
U3	C/parent
U4	B/r

ACL vs. Capabilities

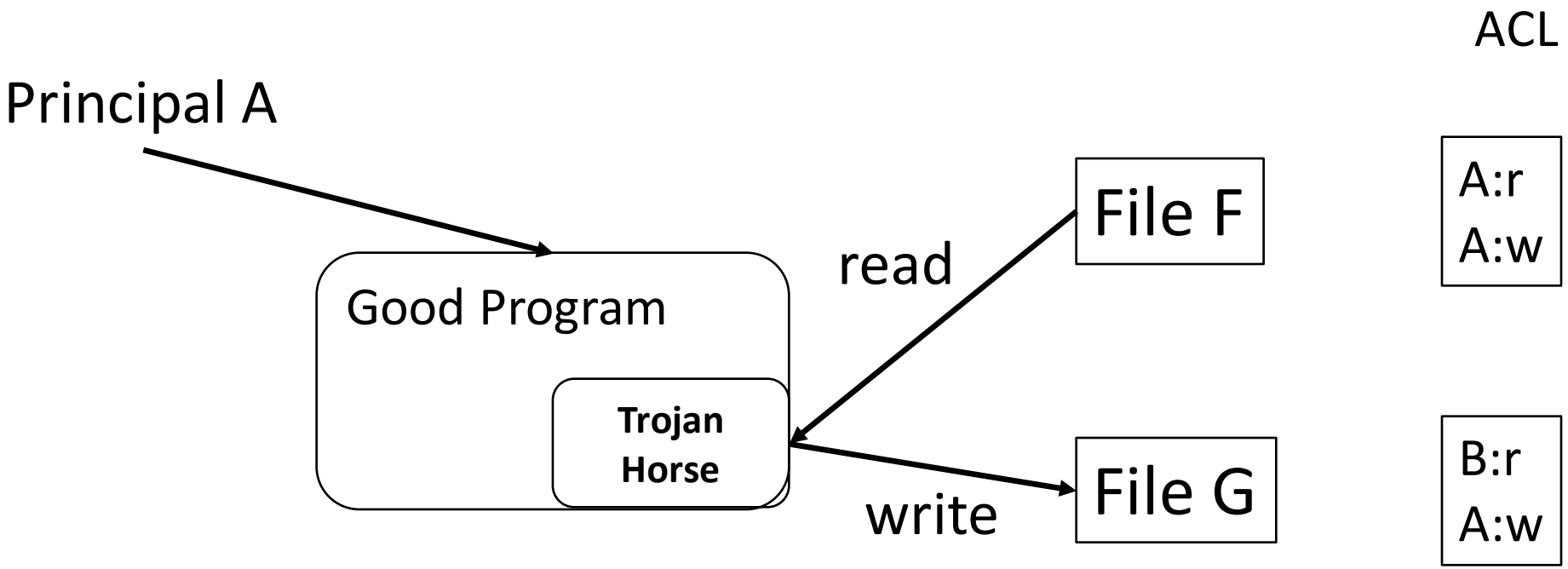
- ACL require authentication of subjects
- Capabilities do not require authentication of subjects, but do require unforgeability and control of propagation of capabilities. Usually implemented trough cryptography.

- The Confused Deputy Problem [1986]
  - Example: Cross-Site Scripting (XSS)
  - Solution: Use Capabilities Implementation

DAC Problems

- The underlying philosophy in DAC is that subjects can determine who has access to their objects.
  - There is a difference, though, between trusting a person and trusting a program.
- The copies of file are not controlled
- The Trojan Horse attack [1970]
  - Solution: use MAC ☺

Trojan Horse attack



Principal B cannot read file F

Buggy software can become Trojan Horses

- When a buggy software is exploited, it executes the code/ intention of the attacker, while using the privileges of the user who started it

- This means that computers with only DAC cannot be trusted to process information classified at different levels

Mandatory access controls

Modeling Access Control

- Assigning access rights based on regulations by a central authority
- Implemented using a *"reference monitor"*
  - Small Trusted Computing Base (TCB) [John Rushby, 1981, OSP]
- Implemented using Virtualization
- TOCTTOU problem
  - Time of check to time of use

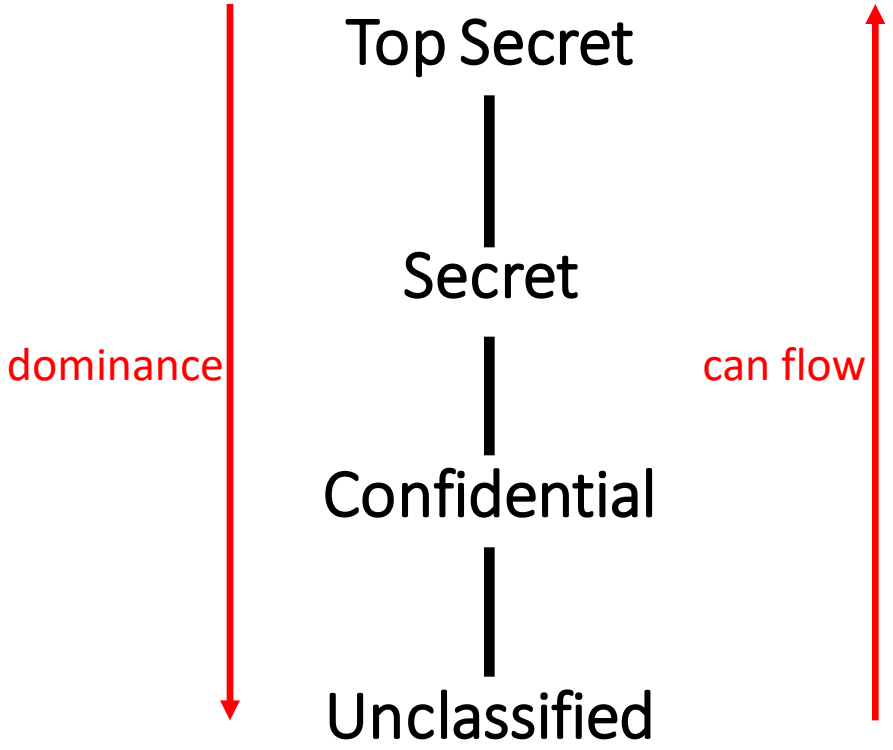
Modeling Access Control

- Multi-level security (MLS)
  - Bell-Bell-LaPadula (BLP)
  - Biba Model
- Chinese Wall

Multi-level security (MLS)

- The capability of a computer system to carry information with different sensitivities

- Bell-LaPadula (BLP) Model [1973]
- Biba Model



BLP Model

- Aims to capture confidentiality (read) requirements only
- The system is modelled as transitions through a set of states, starting from an initial state.
  - State = Object, access matrix, current access information
- State transition rules describe how a system can go from one state to another
- Each subject s has a maximal security level Lm(s), and a current security level Lc(s)
- Each object has a classification level

BLP Model

- A state is secure if:
  - A) Simple Security Property (SS): no subject may read data at a higher level
  - B) The *-Property (SP): no subject may write data at a lower level
    - (due to the fear of Trojan Horse)

- A system is secure if and only if every reachable state is secure.

BLP Problems

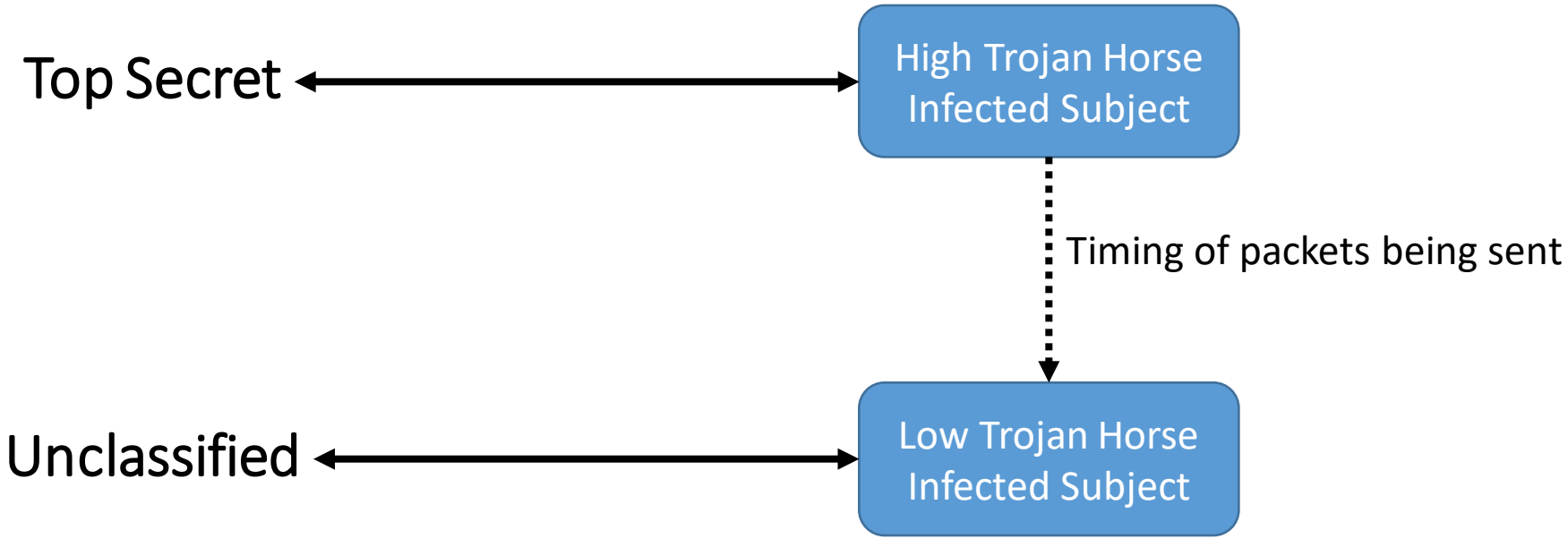
- Consider a system with subjects s1, s2, and objects o1, o2
  - Lm(s1) = Lc(s1) = L(o1) = high
  - Lm(s2) = Lc(s2) = L(o2) = low
- And the following execution
  - s1 gets access to o1, reads something, releases access, then changes current level to low, gets write access to o2, writes to o2
- Every state is secure, yet illegal information exists
- Solution: **tranquility** principle: subject cannot change current levels, or cannot drop to below the highest level read so far

BLP Problems

- There is no ACK from High to Low
- Not all system components can be enforced by BLP, e.g., memory management must have access to all levels
  - Called *“trusted subjects”*
- Can overwrite high and more important files

BLP Problems

- Covert channels cannot be blocked by star-property



Biba Model

- Integrity is also very important
- Each subject (process) has an integrity level; Each object has an integrity level ; Integrity levels are totally ordered

- NO read down; NO write up
  - BLP upside down

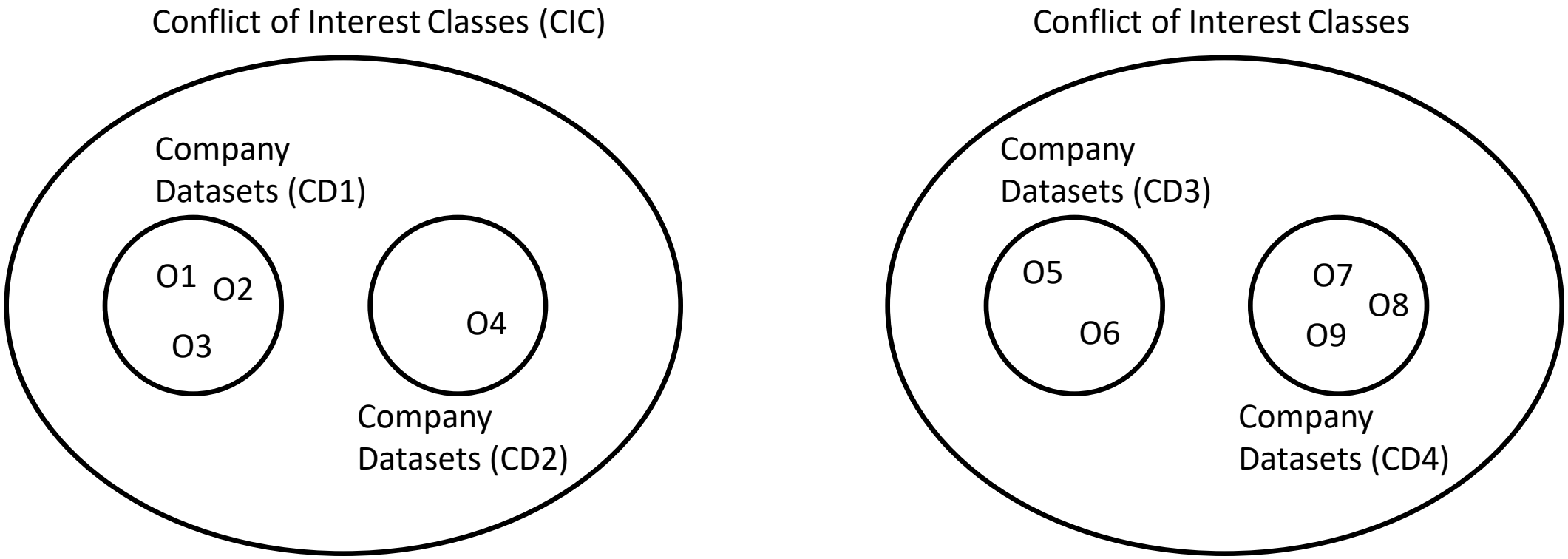
- The integrity of an object is the lowest level of all the objects that contributed to its creation

Biba Model

- Used by Windows

- E.g., A Internet Explorer Browser can download a file (with Low priority) and read everything in the system. It cannot write to it.

Chinese Wall [1989]



Chinese Wall

- S can read O only if
  - O is in the same company dataset as some object previously read by S (i.e., O is within the wall)

  or
  - O belongs to a conflict of interest class within which S has not read any object (i.e., O is in the open)
- S can write O only if
  - S can read O by the simple security rule

  and
  - no object can be read which is in a different company dataset to the one for which write access is request

Chinese Wall

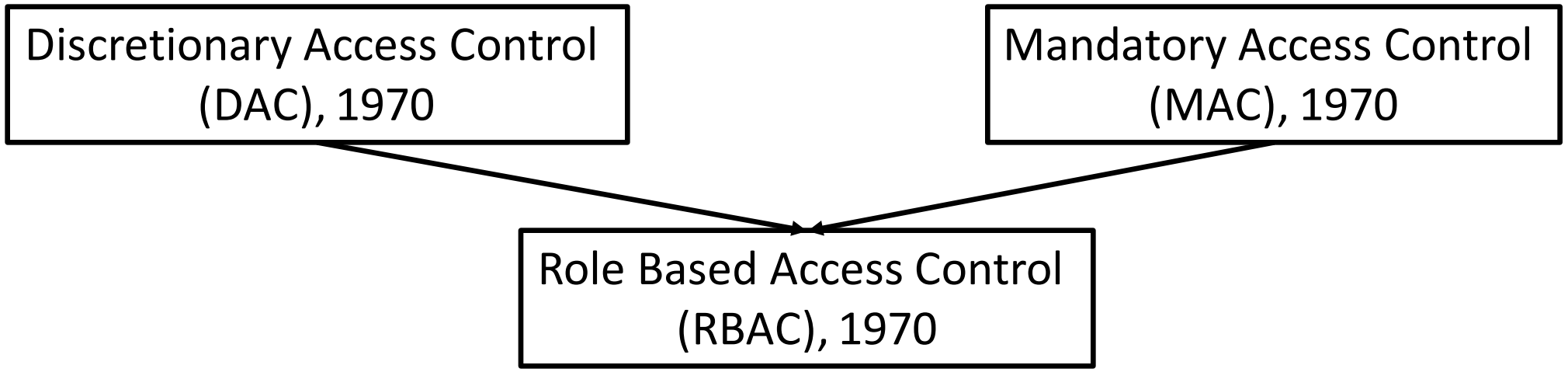
- Once a subject reads two objects from different CDs, that subject may never write any object.

- S1 reads information from an object in CD1.
- S1 writes that information to object O6 in CD3.
- S2 reads that information from O6.
- At the end of this sequence, S2 would have read information pertaining to both CD1 and CD2, which would violate the Chinese Wall policy since both CDs are in the same CIC.

Role-Based Access Control

Role-Based Access Control

- In the real world, security policies are dynamic.
- E.g., a user promotes at his job, therefore his rights must change (deleted, added, etc.)



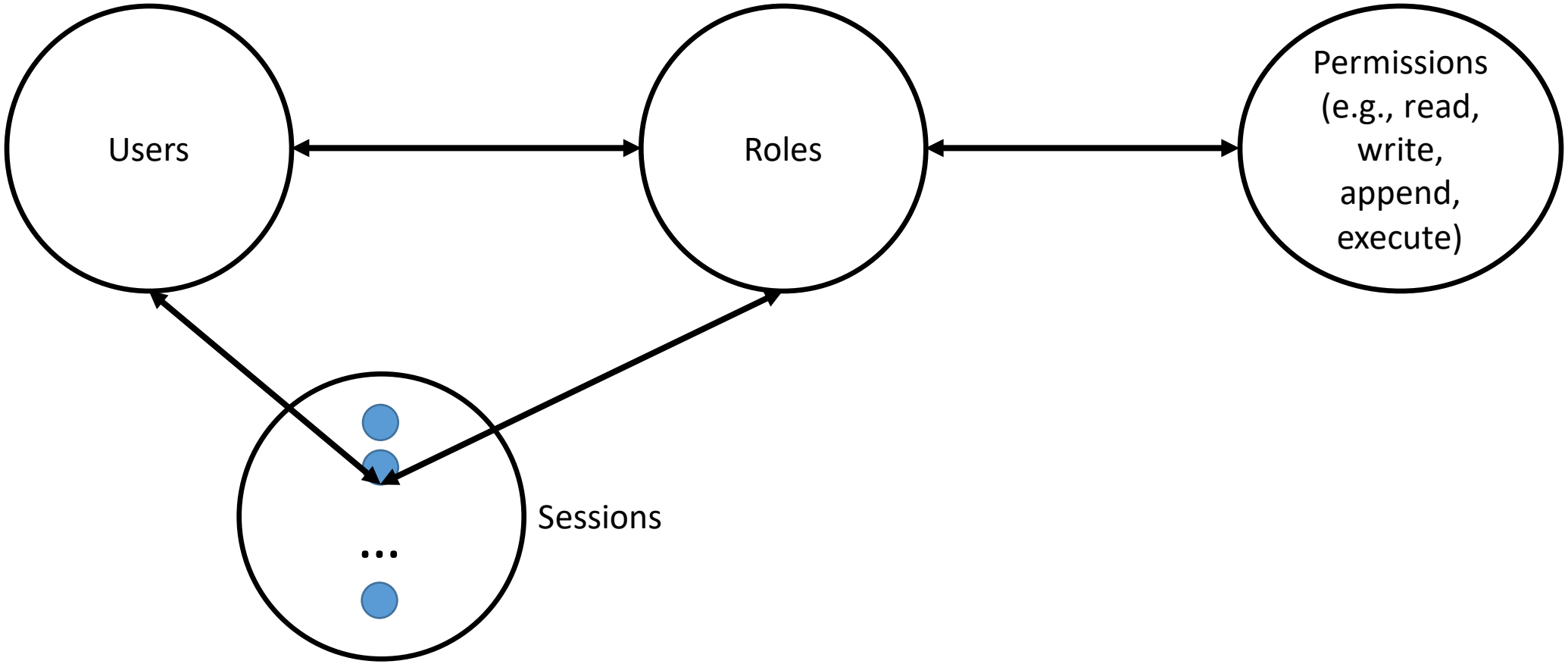
Role-Based Access Control

- Can be configured to do MAC
  - roles simulate clearances (ESORICS 96)
- Can be configured to do DAC
  - roles simulate identity (RBAC98)

Role-Based Access Control

- Changes the underlying subject--object model
  - a policy is a relation on roles, objects, and rights
- Subjects are now assigned to roles;
  - *role assignment*
- Roles are hierarchical

Role-Based Access Control



Roles as policy

- A role brings together
  - a collection of users and
  - a collection of permissions a collection of permissions
- These collections will vary over time
- A user can be a member of many roles
- Each role can have many users as Each role can have many users as members

Implementation

- Using a access matrix

RBAC Shortcomings

- Role granularity is not adequate leading to role explosion
- Role design and engineering is difficult and expensive
- Assignment of users/permissions to roles is cumbersome
- Adjustment based on local/global situational factors is difficult

Future Access Control

Attribute-Based Access Control (ABAC)

- Attributes are name:value pairs
  - possibly chained
  - values can be complex data structures
- Associated with
  - users
  - subjects
  - objects
  - contexts
- Converted by policies into rights just in time
  - policies specified by security architects
  - attributes maintained by security administrators
  - ordinary users morph into architects and administrators

Resources

- [Sandhu, 2006] http://www.profsandhu.com/confrnc/asiaccs/asiaccs06-pei.pdf
- http://www.cs.cornell.edu/courses/cs5430/2011sp/NL.accessControl.html
- http://cnitarot.github.io/courses/cs526_Spring_2015/s2014_526_ac.pdf

Operating Systems Security

Asst. Prof. Mihai Chiroiu

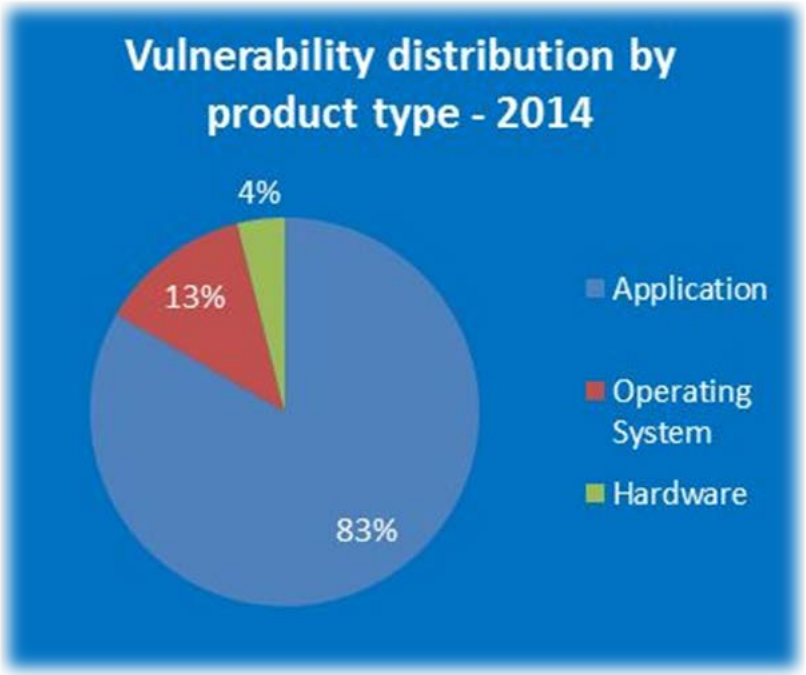
SYSTEMS LABORATORY
Computer Science & Engineering Department

OS principles

- hardware abstraction
- resource management: accounting, scheduling, and synchronisation
- storage and communication services: file systems, network, inter-process communication (IPC)
- libraries of common functions: libc
- management of user interaction and interface

- More here: http://ocw.cs.pub.ro/courses/so

Stats (2014)



Operating system	# of vulnerabilities	# of HIGH vulnerabilities	# of MEDIUM vulnerabilities	# of LOW vulnerabilities
Apple Mac OS X	147	64	67	16
Apple iOS	127	32	72	23
Linux Kernel	119	24	74	21
Microsoft Windows Server 2008	38	26	12	0
Microsoft Windows 7	36	25	11	0
Microsoft Windows Server 2012	38	24	14	0
Microsoft Windows 8	36	24	12	0
Microsoft Windows 8.1	36	24	12	0
Microsoft Windows Vista	34	23	11	0
Microsoft Windows RT	30	22	8	0

http://www.gfi.com/blog/most-vulnerable-operating-systems-and-applications-in-2014/

What should the OS protect?

- Itself (from users)
- Processes (both services and user's application)
- Files access
- Communication (both IPC and network)

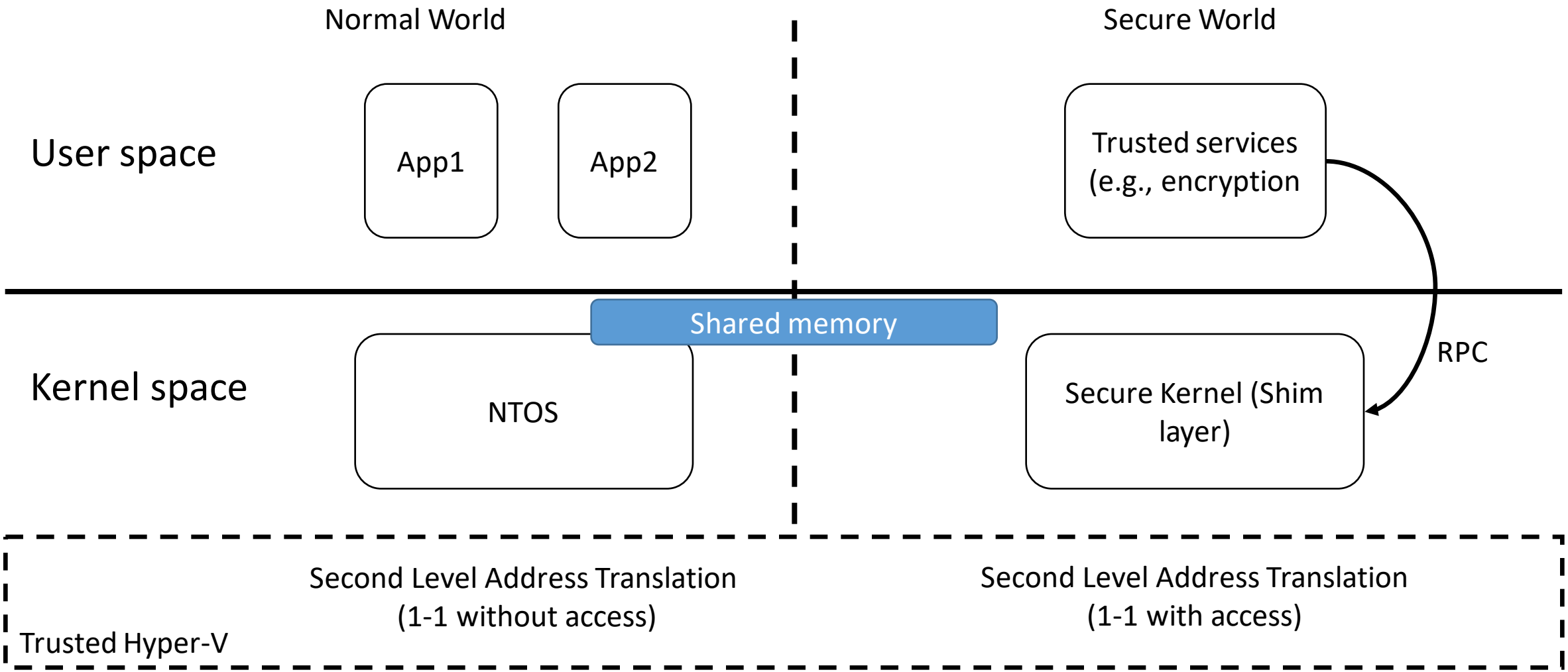
First, authentication

- Most common technique are passwords (i.e., something you know)
  - Stored as hashes typically using a random *salt*
- Tokens (i.e., something you have)
  - Using HSM
  - Often combined with a PIN
- Biometrics (i.e., something you are)
  - Fingerprints, iris scans, etc.

- We will assume that authentication is validated!

Windows 10

Virtualization-based security (VBS)



Code Integrity

- Kernel Mode Code Integrity (KMCI)
  - Validate drivers' signature
- User Mode Code Integrity (UMCI)
  - Validate apps signature
- AppLocker
  - Policy for what applications can be executed

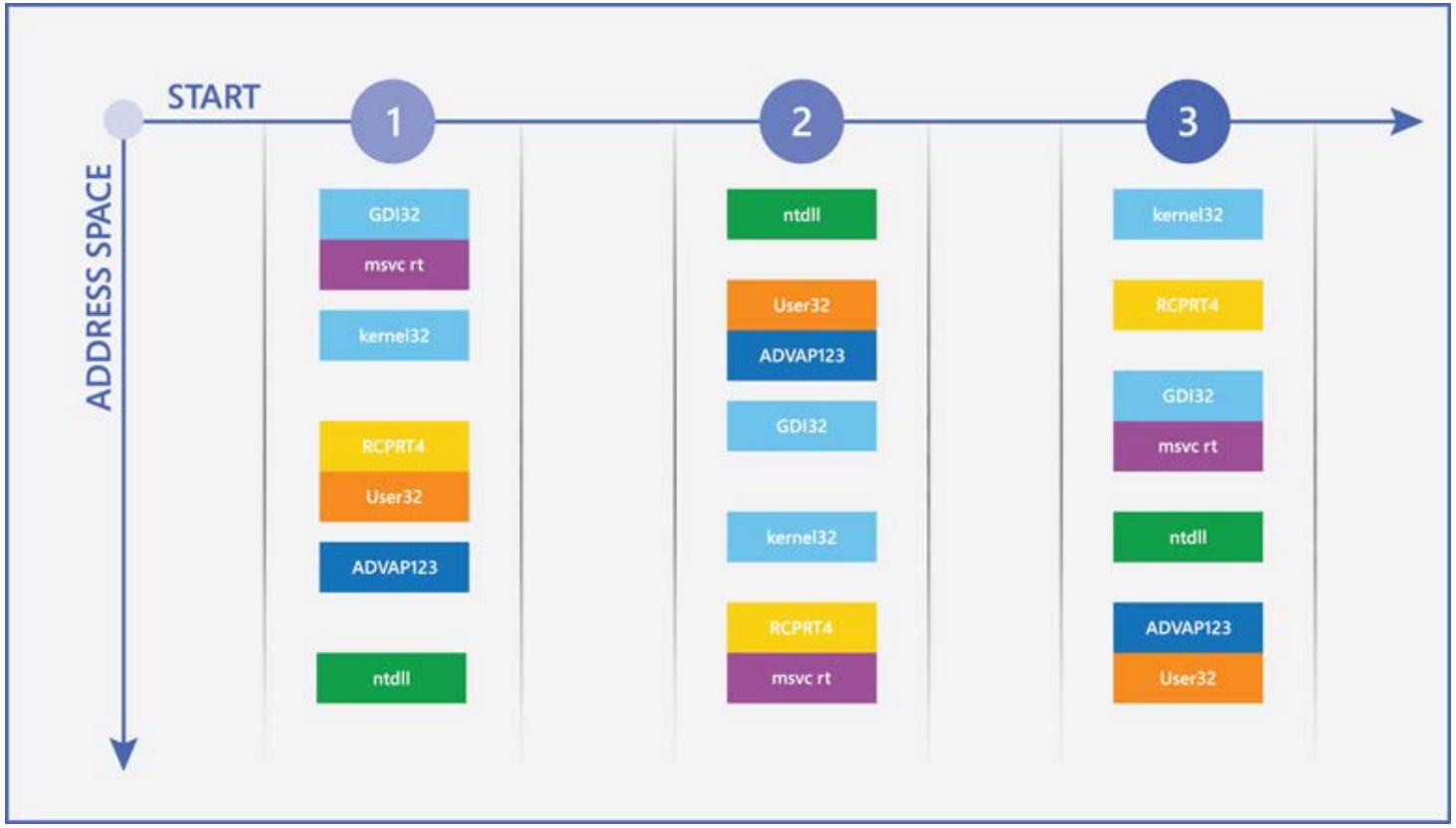
Protected Processes

- Windows 10 prevents untrusted processes from interacting or tampering with those that have been specially signed.
- Protected Processes defines levels of trust for processes.
- Less trusted processes are prevented from interacting with and therefore attacking more trusted processes.

Address Space Layout Randomization (ASLR)

- Present in most OSes
- Not a real solution

(part of a complex one) [1]



ASLR implementation

- On Windows, ASLR does not affect runtime performance, but it can slow down the initial loading of modules.
  - ASLR also randomizes heap and stack memory
- On Linux, ASLR imposes 26% [9]
- On Android, ASLR bases for all others and the bases remain constant across executions [10]
- On iOS, dyld_shared_cache (libraries) load address is randomized (at boot time) [11]
- ASLR cannot be force-enabled for applications on Linux (they must be compiled with PIE), as EMET can do on Windows.

Data Execution Prevention (DEP)

- DEP uses the No eXecute bit on modern CPUs
- Available on all major Oses
- Not real use if you can access mprotect/VirtualProtect/etc.

TrueCrypt - Full-disk encryption (3rd party)

- Password used to encrypt/decrypt when mounting the partition.
- Supports plausible deniability
  - can be configured to hide even the existence of encrypted data.
  - Unused space on an encrypted partition is initialized with random data, encrypted volume is indistinguishable from such random data.

BitLocker – Full-disk encryption

- Encrypting entire hard drives
- Support for Self-Encrypting Drives (SED) for offloading encryption
- Uses Trusted Platform Module (TPM) v1.2 to validate pre-OS components



Where's the Encryption Key?

1. SRK (Storage Root Key) contained in TPM
2. SRK encrypts FVEK (Full Volume Encryption Key) protected by TPM/PIN/USB Storage Device
3. FVEK stored (encrypted by SRK) on hard drive in the OS Volume

© Mihai Chiroiu

SYSTEMS LABORATORY

Computer Science & Engineering Department

File permissions

- Stored as an ACE in a discretionary access control list (DACL) that is part of the object's security descriptor.
- Permissions can also be explicitly denied.
- Inherited permissions are those that are propagated to a child object from a parent object.

Network access

- Per application firewall

Microsoft Bounty Programs

- Online Services Bug Bounty (Microsoft Azure services additions: 22nd April 2015)
  - $500 USD up to $15,000 USD.
- Mitigation Bypass Bounty (Windows 10)
  - up to $100,000 USD
- Bounty for Defense (Windows 10)
  - up to $100,000 USD

- https://technet.microsoft.com/en-US/security/dn425036

Linux

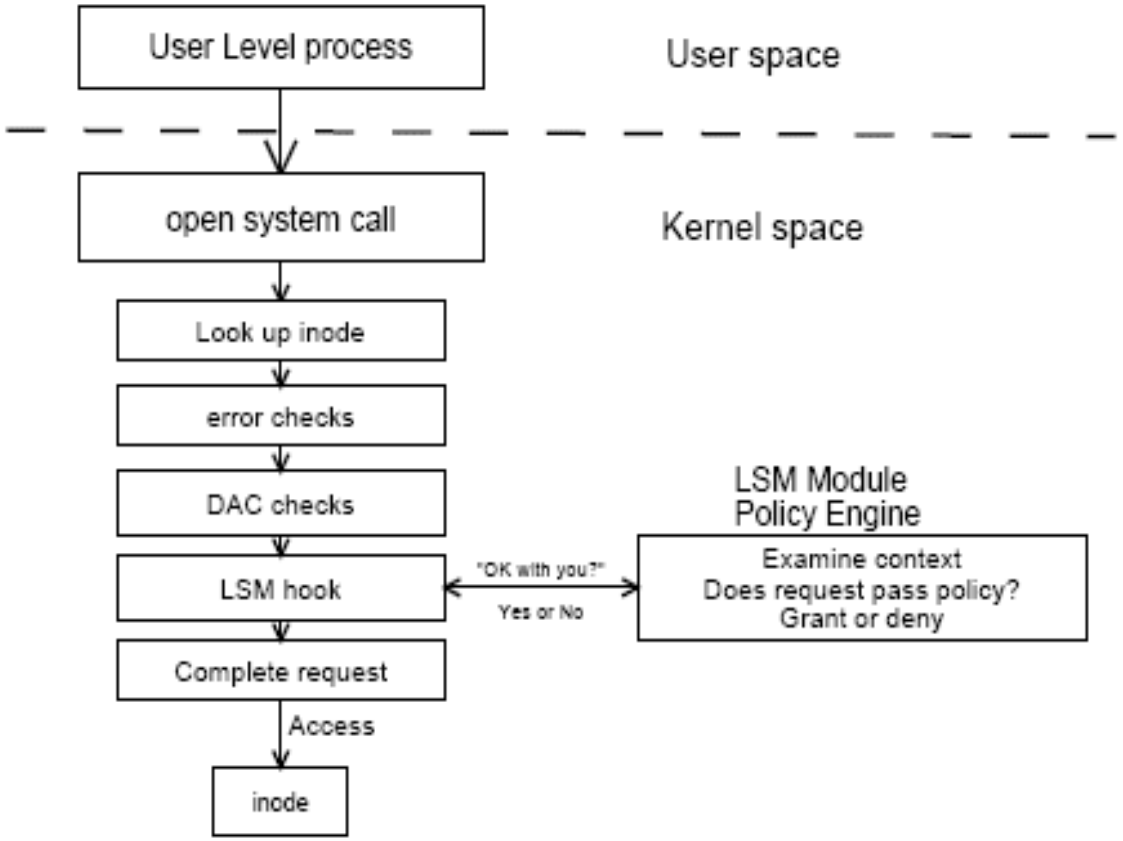
Linux - *setuid*

- Sometimes we want to specify that a file can only be modified by a certain program.
- Thus, we want to control access on a per-program, rather than a per-user basis.
- We can achieve this by creating a new user, representing the role of a modifier for these files.
- Mark the program, as *setuid* to this user.
- This means, no matter who started the program, it will run under the user id of this new user.

LUKS – Full-disk encryption [3]

- A master key is generated by the system (used to encrypt/decrypt data on disk)
- Protected using the user's password
- Several master keys are stored, one for each user

Linux Security Modules (2002) [6]



- IPC Hooks
- Filesystem Hooks
- Network Hooks

SELinux

- Mandatory Access Control system for Linux
- Implement Flask architecture [7]

- A process (a daemon or a running program) is called a *subject*.
- A role defines which users can access that process.
- An *object* in SELinux is anything that can be acted upon
- A file's context is called its *type* in SELinux lingo

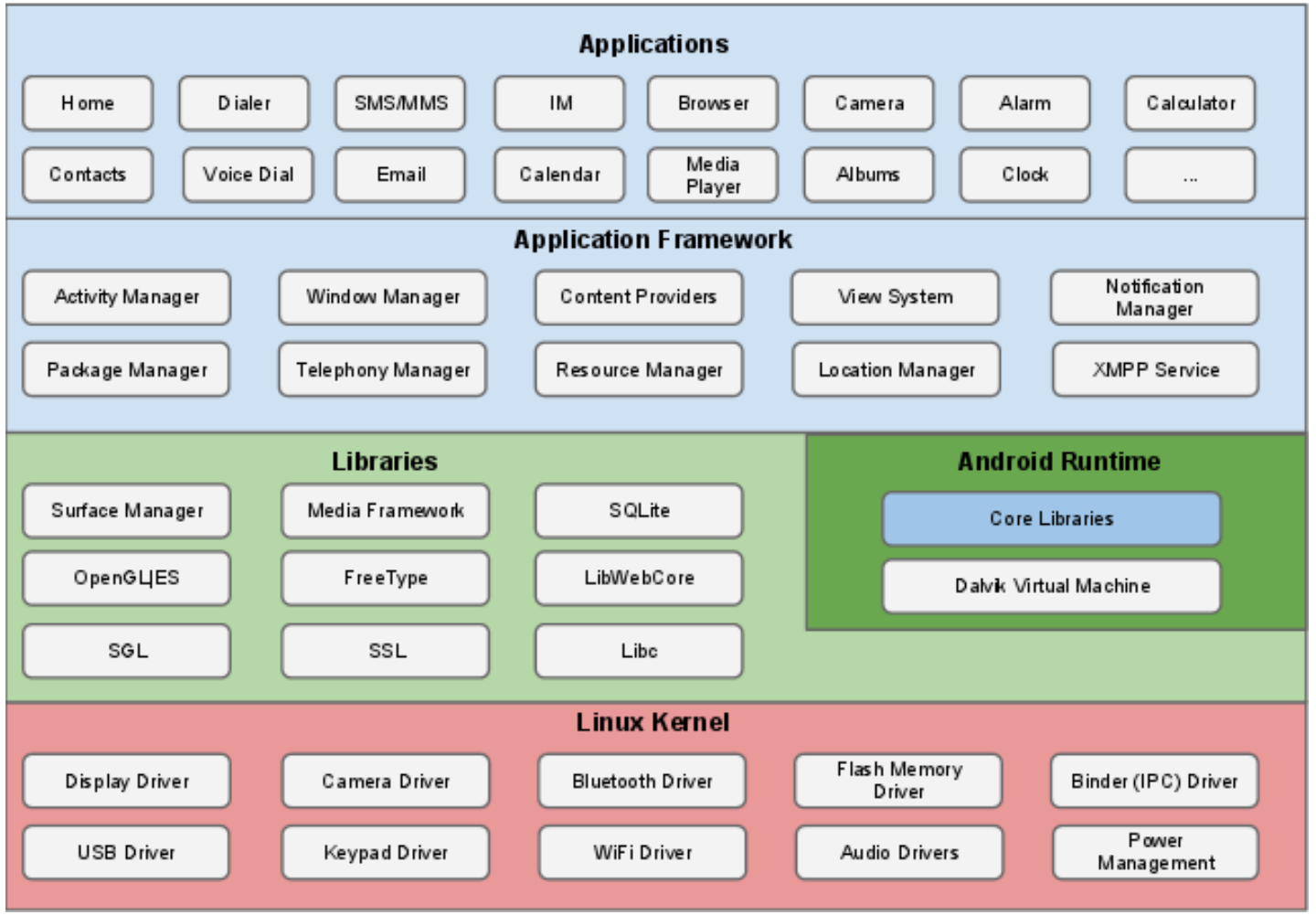
SELinux

- An SELinux policy defines user access to roles, role access to domains, and domain access to types.
- Possible modes are Enforcing, Permissive, or Disabled

- `-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html`
- `system_u:system_r:httpd_t:s0 7126 ? 00:00:00 httpd`
- `sesearch --allow --source httpd_t --target httpd_sys_content_t --class file`
  - `allow httpd_t httpd_sys_content_t : file { ioctl read getattr lock open } ;`

Android

Android Architecture



Package (APK) integrity

- Components of applications
  - Activity: User interface
  - Service: Background service
  - Content Provider: SQL-like database
  - Broadcast receiver: Mailbox for broadcasted messages
- META-INF contains the application certificate and package manifest
- Certified by developer
- Used for: application upgrade; application modularity (two apps from same developer can collude);

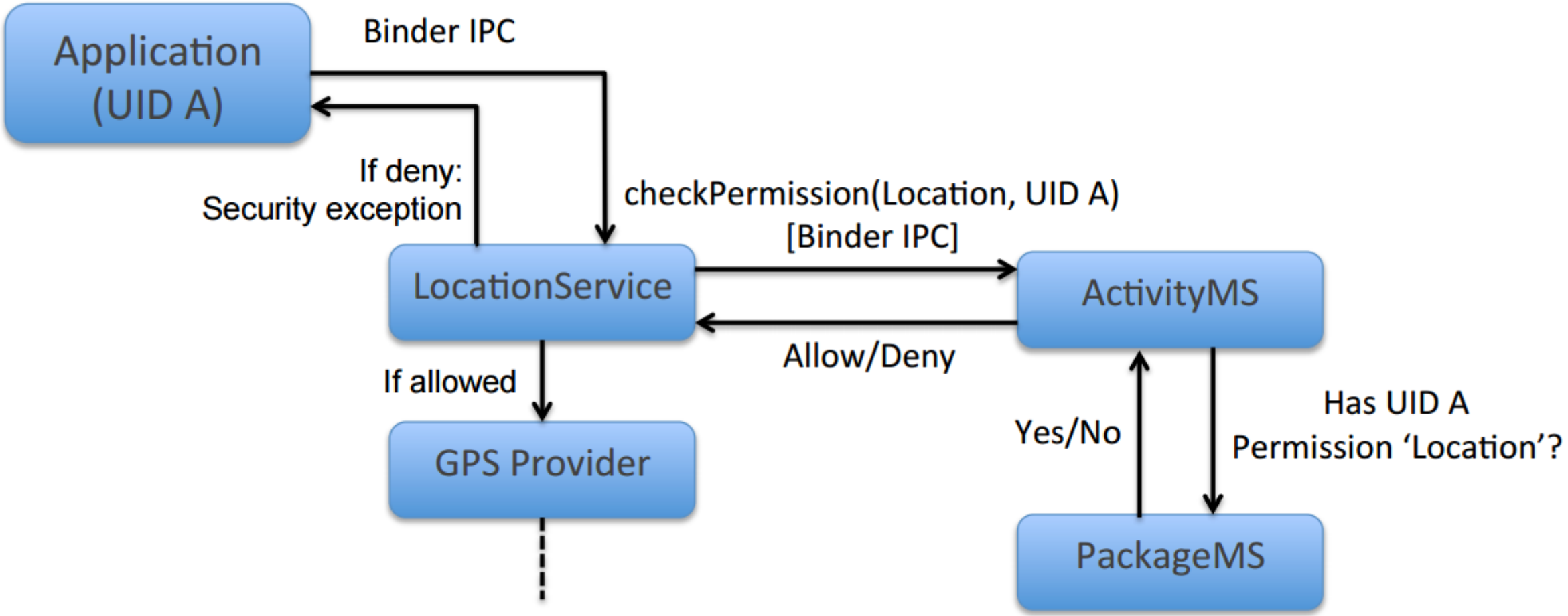
Android Security Basics

- Applications, by default, have no permissions
- Applications statically declare the permissions they require
  - Android system prompts the user for consent at the time the application is installed
  - No mechanism for granting permissions dynamically (at run-time)
  - In AndroidManifest.xml, add one or more <uses-permission> tags
  - e.g., <uses-permission android:name= "android.permission.RECEIVE_SMS" />

Android Sandbox

- Each application is isolated in its own sandbox
  - Applications can access only its own resources
  - Access to sensitive resources depends on the application's rights
- Enforced by underlying Linux Kernel (SELinux) and middleware
- Each App is assigned a unique UserID during installation and runs in separate process

Android Sandbox



Android Sandbox

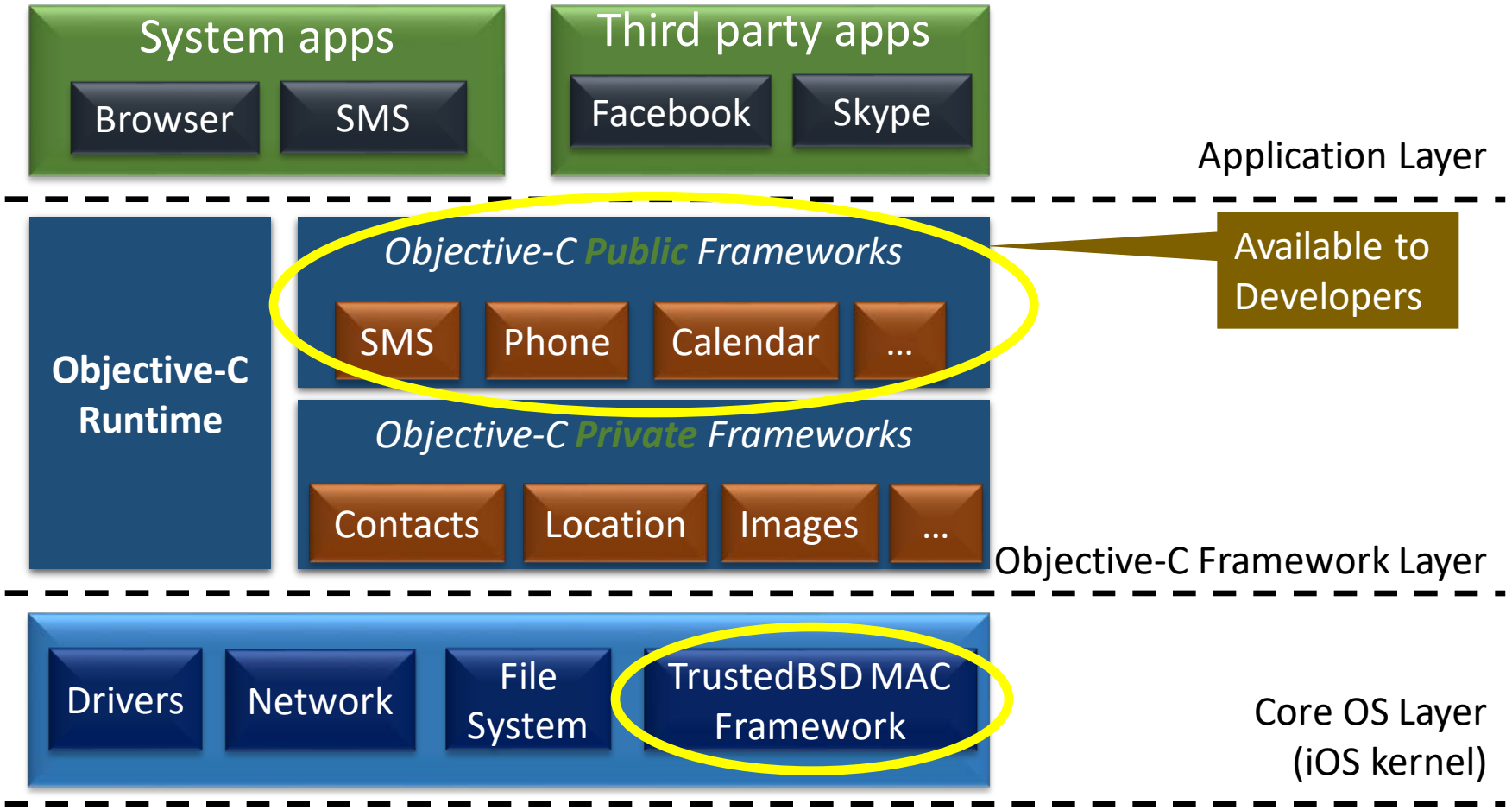
- App UID must be member of a Linux group to have access to sockets, etc.
- UID of an app with corresponding permission is added to group during install
- Kernel access errors translated into Java security exceptions by core libraries

Isolated Processes

- Security-aware application developer can declare in application manifest that a Service component should be executed as an isolated process
  - Component executed on separate process with UID nobody
  - Nobody is a UID with no privileges
    - All permission checks will return deny
    - No file system access
  - only communication with it is through the Service API
- Allows compartmentalization of the app

iOS

iOS Architecture



iOS Protection Mechanisms

- Encrypted file system
- Applications signing
- Vetting processs (app reviewing)
  - 700 - 1000 apps are submitted each day [Apple]
- Address Space Layout Randomization (ASLR)
- Non-executable memory security model (with code signing on memory pages)

Sandboxing

- Enforcement at the Objective-C runtime layer
  - That could be bypassed
- Enforcement by the TrustedBSD kernel module
  - Based on a generic profile that forces application containment (for IPC and files)
- Custom rules added by users are allowed

Xen VMM

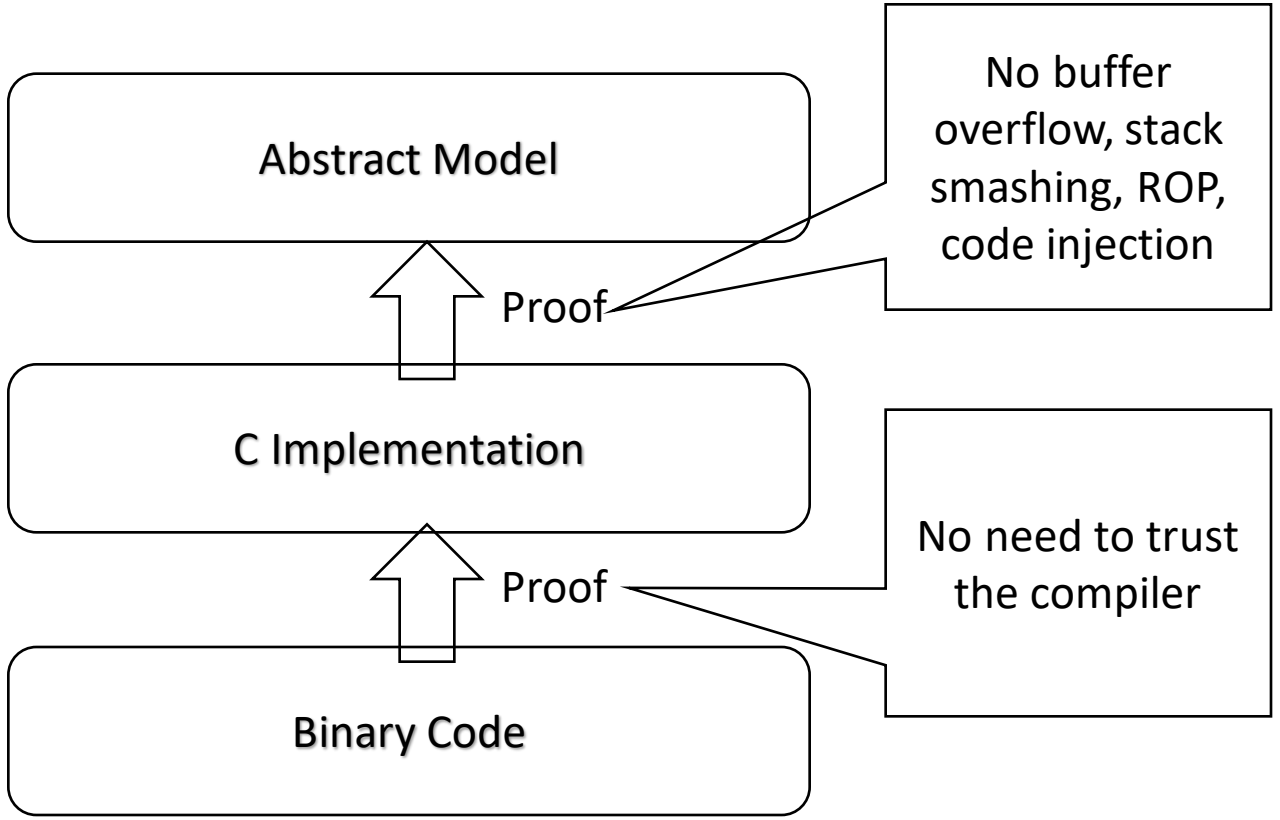
Security possibilities

- VM introspection
- Dom0 dissagregation
  - Driver domains
- Xen Security Module (same as LSM)
  - Restricts hypercalls to those needed by a particular guest

Formally verified security kernel

seL4 [4]

- Based on a minimal L4 kernel (drivers are outside kernel, user-mode processes)
- A refinement proof establishes a correspondence between a high-level (abstract) and a low-level (concrete, or refined) representation of a system.



References

- [1] https://www.trust.informatik.tu-darmstadt.de/fileadmin/user_upload/Group_TRUST/PubsPDF/jit-rop.pdf
- [2] https://technet.microsoft.com/en-us/library/mt601297(v=vs.85).aspx
- [3] https://gitlab.com/cryptsetup/cryptsetup/wikis/LUKS-standard/on-disk-format.pdf
- [4] http://web1.cs.columbia.edu/~junfeng/09fa-e6998/papers/sel4.pdf

References

- [5] https://opensource.com/business/13/11/selinux-policy-guide
- [6] https://www.usenix.org/legacy/event/sec02/full_papers/wright/wright.pdf
- [7] https://www.nsa.gov/research/_files/publications/flask.pdf
- [8] http://css.csail.mit.edu/6.858/2012/readings/android.pdf
- [9] http://nebelwelt.net/publications/files/12TRpie.pdf

References

- [10] https://copperhead.co/blog/2015/05/11/aslr-android-zygote
- [11] http://antid0te.com/CSW2012_StefanEsser_iOS5_An_Exploitation_Nightmare_FINAL.pdf

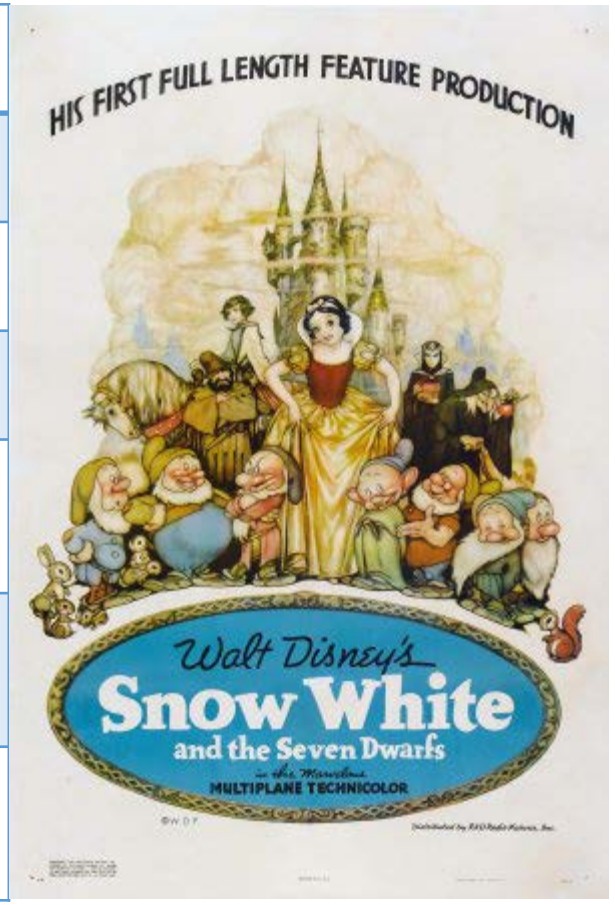
Network Security

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

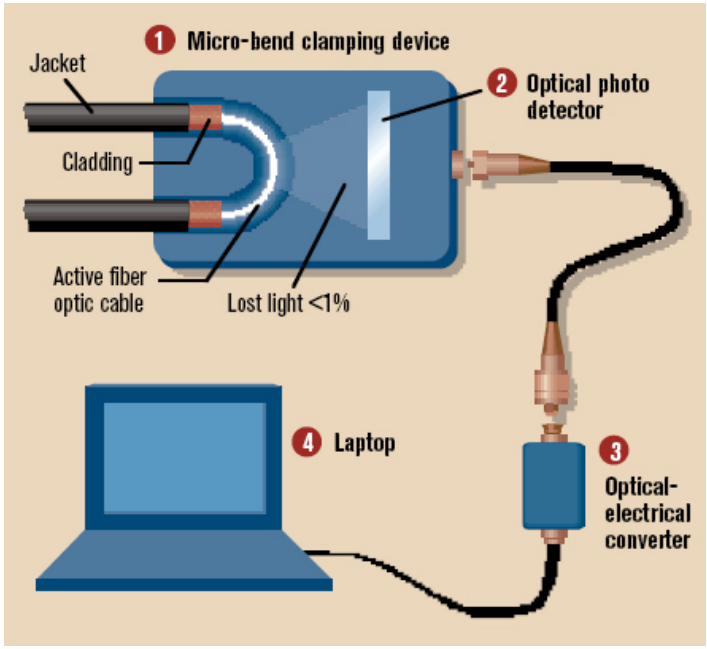
Network 7 Layers [1]

Sleepy	Physical	The group new that physical connections are boring, and figured it might as well assign the physical layer to dwarf ``Sleepy''.
Sneezy	Link	If you monitor a network and watch the pattern of packets emitted by a computer, you'll immediately understand the relationship between link-layer protocols and ``Sneezy''.
Happy	Network	Everyone's happy with the network layer. Well... to be honest, the only network layer protocol that makes everyone's happy is the Internet Protocol.
Doc	Transport	This one's obvious -- it definitely takes a Ph.D. to understand the subtleties of a transport layer protocol.
Dopey	Session	Yep, even the designers realized that having a separate session layer is a dopey idea. They decided to follow Disney's approach of adding comic relief, so they stuck in a completely unnecessary layer and laughed about it.
Bashful	Presentation	The designers realized that sooner or later someone would create a presentation layer protocol. However, the group decided to classify such protocols as too ``bashful'' to appear in public. So, even if a presentation protocol is produced, no one gets to see it.
Grumpy	Application	Programmers who design network applications are incredibly grumpy -- they complain about the efficiency of other layers [...]. And users add to the grumpiness, [...] ,they only complain about applications.



Intercepting at physical

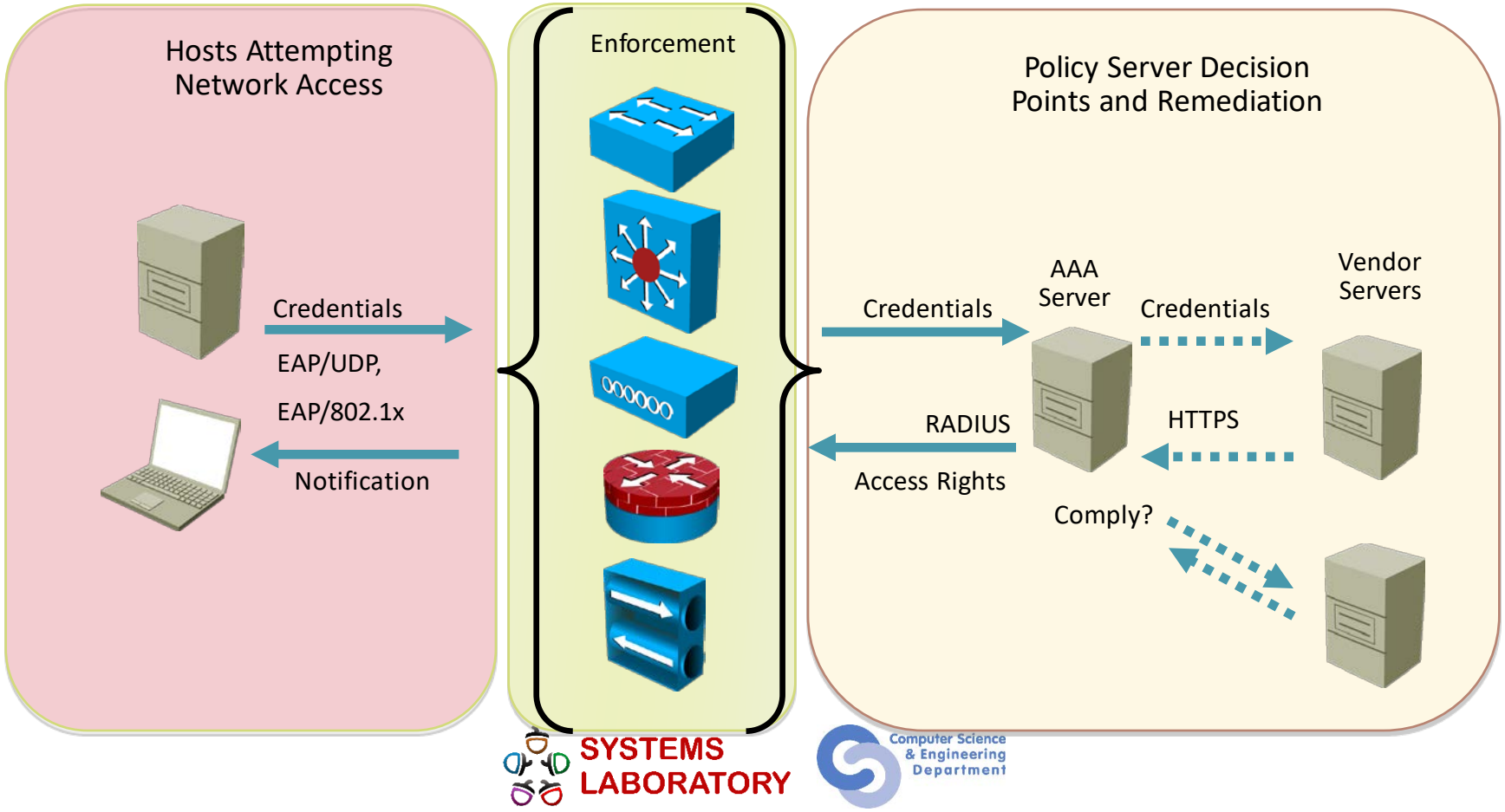




- You can buy one of these for ~350$
- You can detect an attack like this by loss of light (must be lower than 2% in an acceptably quality implementation)

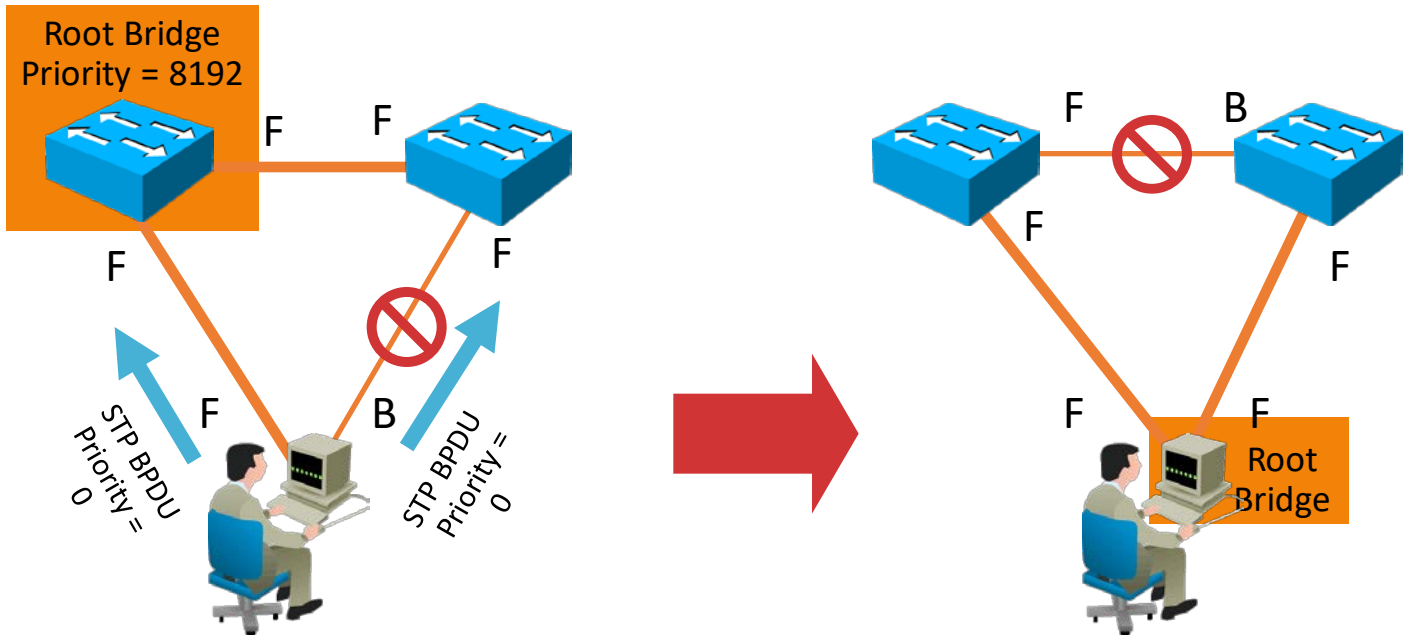
802.1x

- Several devices enforcing different security policies.



L2 Security

- MITM attacks from insiders (ARP Spoofing)
- STP can be broken
- VLAN hopping



L2 Security

- DHCP bindings
- Sticky MAC
- BPDU Guard

VPN topologies

- Remote-access VPNs
  - Remote users must have a broadband Internet connection
  - The VPN parameters are dynamically negotiated
  - The tunnel is established only when required

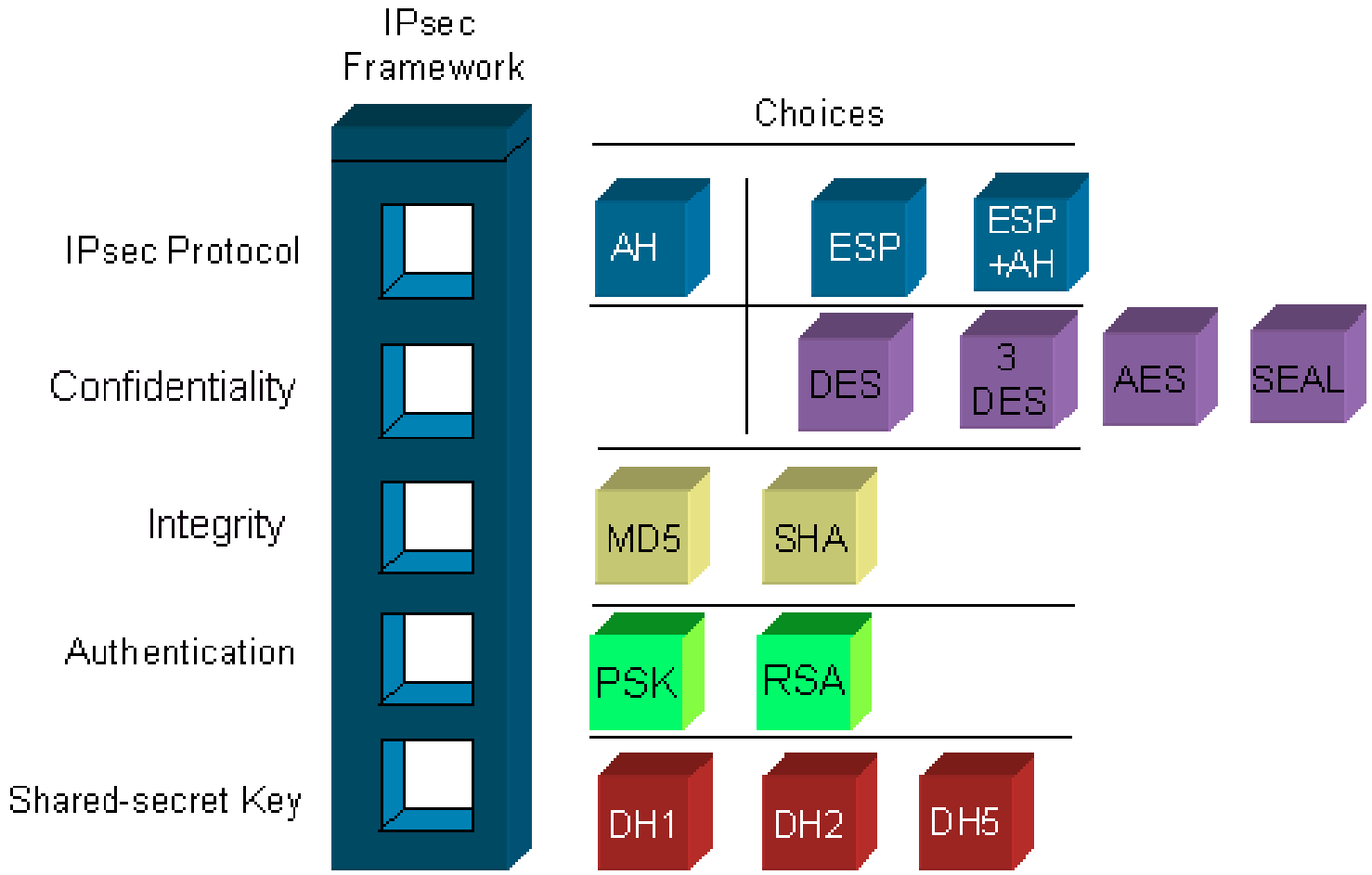
- Site-to-site VPNs
  - Configured between two VPN-aware devices on both ends
  - Always-on
  - Provides interconnectivity between multiple networks on both sites.
  - Each end of the tunnel acts as a gateway for its networks.

IPsec (simplified)

- IPsec is an IETF standard (RFC 2401-2412)
  - Defines ways to deploy VPNs using the IP addressing protocol.
  - Is a framework of open standards that describe how to secure communication.

- Relies on existing algorithms to provide:
  - Encryption
  - Authentication
  - Data integrity
  - Secure key exchange
- Can work over any L2 connection.

IPsec (simplified)



© Mihai Chiroiu

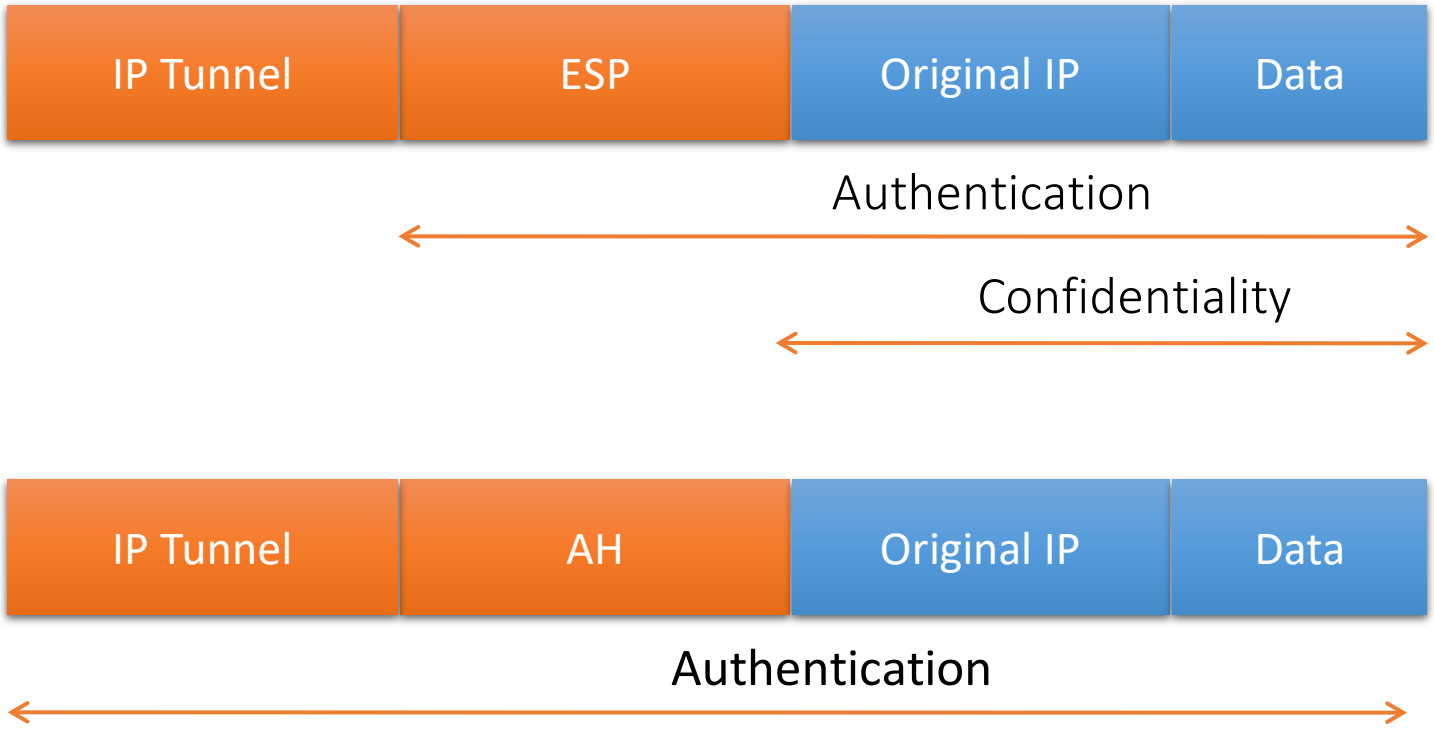
SYSTEMS LABORATORY

Computer Science & Engineering Department

IPsec (simplified)

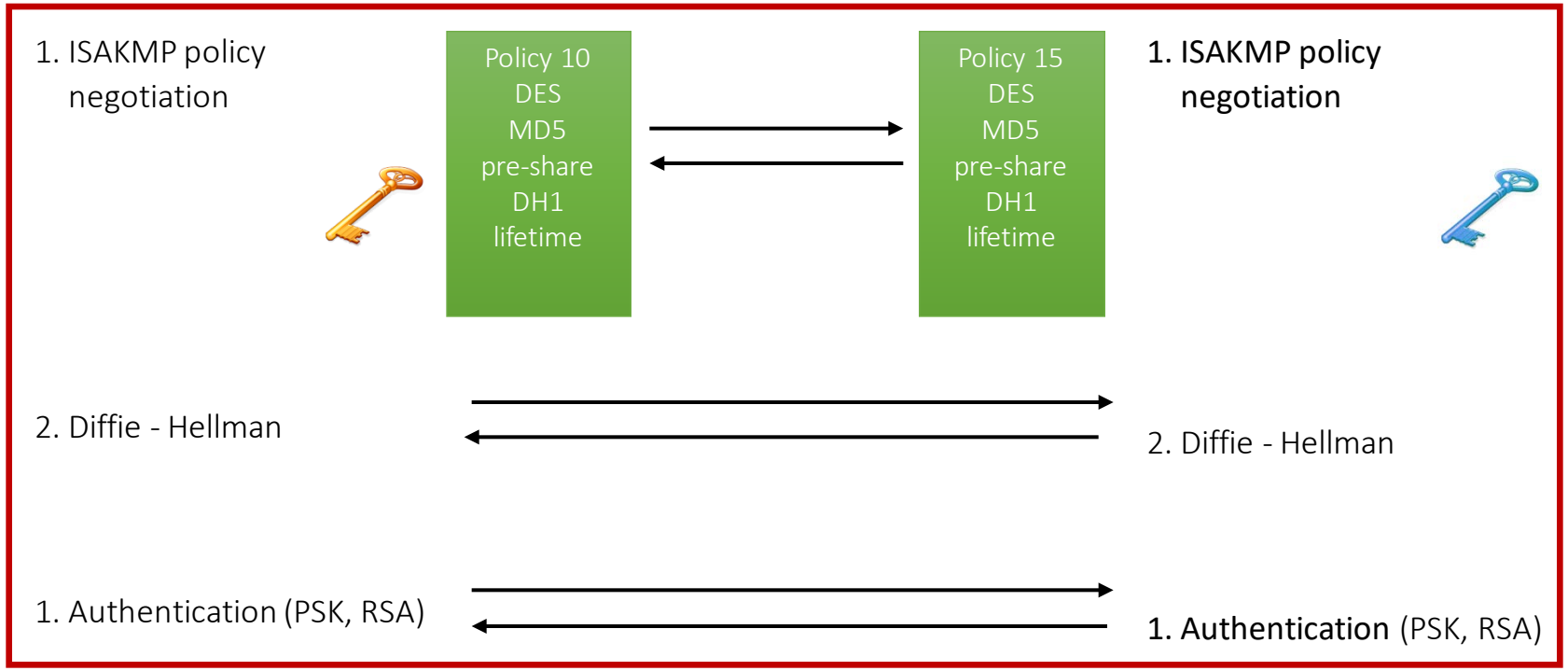
- Algorithms that provide confidentiality (encryption):
  - Examples: DES, 3DES, AES, SEAL
- Algorithms that ensure integrity:
  - Examples: MD5, SHA, along with other versions
- Algorithms that define the authentication method:
  - Choices include: pre-shared keys (PSK) or digitally signed using RSA.
- The mechanism to securely communicate a shared key:
  - Several DH (Diffie-Hellman) groups

IPsec (simplified)



IPsec (simplified)

IKE phase 1



IKE phase 2



The NAT problem

- AH hashes the IP header and the TCP header and expects them to remain unaltered.
- NAT(PAT) overwrites the layer 3 and 4 addresses and port numbers.
- How do you solve this?
- Solution: NAT-T (NAT-Traversal or NAT-Transparency)
  - In IKE Phase 1, an unencrypted but hashed message is sent.
  - At destination, if the hashes do not match, there is a NAT router in between.
- NAT-T encapsulates everything (including ESP) in an UDP header
  - There is also a TCP variant available when connection state tracking is required.
    - If an IPS/IDS device is present, for example.

Routing protocols (attacks)

- OSPF
- BGP
  - (Sub)Prefix Hijacking (I'm Youtube now ☺)
- Attacks directed to the equipment (routers)

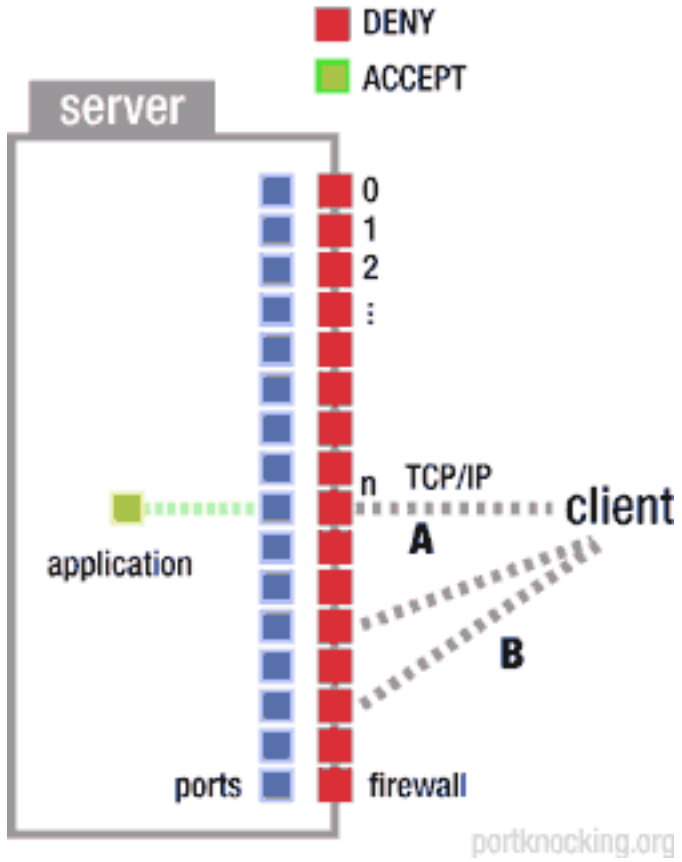
Routing anonymity

- Mixes routers (1981) [6]
  - nodes may reorder, delay, and pad traffic to complicate traffic analysis
- Onion router (1996) [7]
  - produce virtual circuits within link encrypted connections
  - TOR project (2004) [8]
- Crowds (1998) [9]
  - Relay message to random router: with probability p to another router; with probability 1-p, to its intended destination

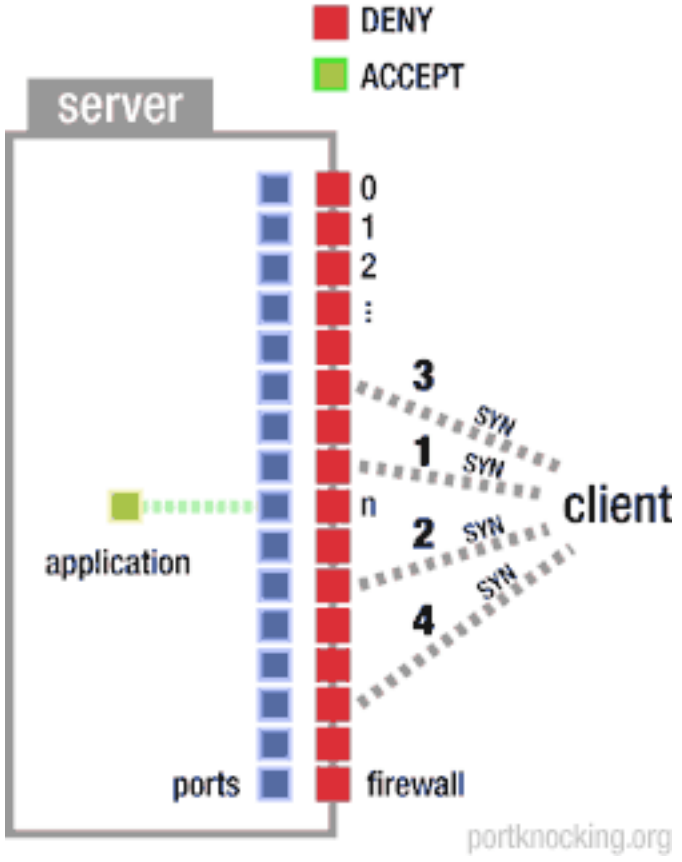
TCP/IP Attacks [4]

- Port scanning
- TCP Sequence Numbers guessing
- Source Routing

Port knocking

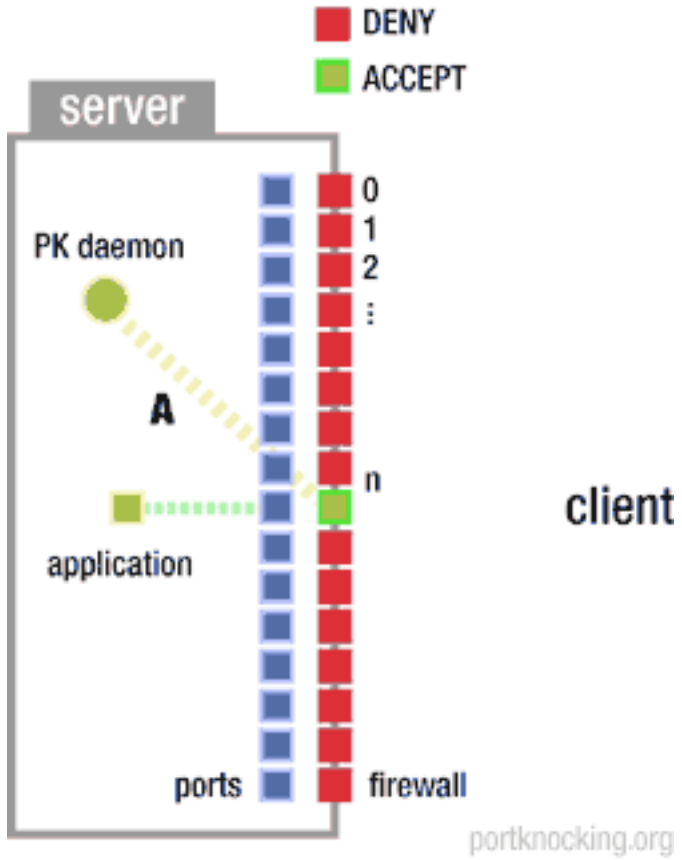


A) The client cannot connect to the application. The client cannot establish a connection to any port.

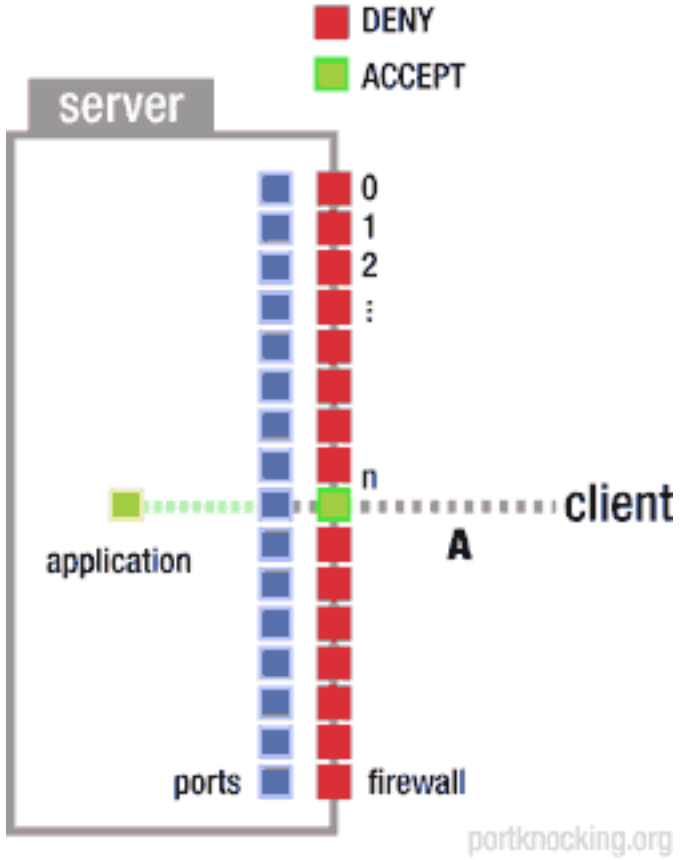


B) The client attempts connection to a number of ports in a predefined sequence. Client receives no ACKs.

Port knocking



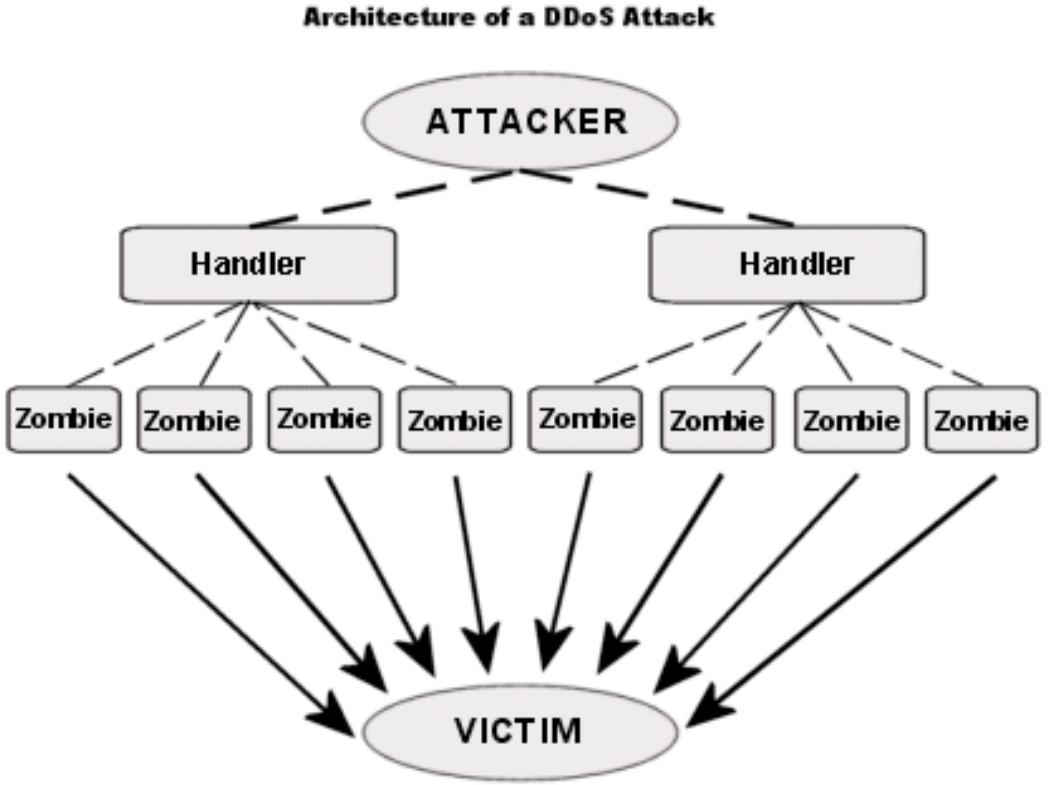
C) The PK daemon interprets the attempts and carries out a task. For example, it opens a specific port (n).



D) The client can now connect to port n.

(Distributed) Denial of Service

- Each network MUST have a benchmark of:
  - Total bandwidth utilization
  - Protocols active in the network
  - Hardware load (For hosts/network devices)
- All the above measured for different times of the day
- These statistics can be used to detect anomalies
- Anomalies can represent attacks
- TCP SYN Flood



SSL

- Developed by Netscape, now an IETF RFC (TLS Jan '99)
- Protocol for using one or two public/private keys
  - to authenticate a sever to a client
  - and by requiring a client key to authenticate the client to the server
  - establish a shared symmetric key (the session key)
- Gives you authentication, message integrity and confidentiality
- Everything except authorization

SSL

- Negotiate the cipher suite
- Establish a shared session key
- Authenticate the server (optional)
- Authenticate the client (optional)
- Authenticate previously exchanged data

- SSL Attacks [10] [11]
- SSL Stripping
- TLS 1.0 wrong crypto (CBC IV's)
- Broken CA (DigiNotar – 2011)

Firewall

- Access control over traffic (at different OSI levels)
- Must be fast
- Whitelisting vs. Blacklisting
- Stateful vs. Stateless
- Next level firewalls: Deep Packet Inspection

Intrusion Detection/Prevention Systems

- Intrusion detection is a classification problem
- Based on signatures (how to be fast? – data structures? GPU?)

Reality	Detection Result		
		True	False
	True	True Positive	False Negative
	False	False Positive	True Negative

Honeypots

- Easy-to-hack environment (hopefully) controlled by administrator
- Used to learn about hackers behavior or as decoy
- Low interaction (emulated) vs. High interaction (real OS/apps)
- Virtual Machines as honeypots

DNS Security [5]

- DNS requests and responses are not authenticated
- DNS relies heavily on caching for efficiency, enabling cache pollution attacks

- DNSSec:
  - Each domain signs their “zone” with a private key
  - Public keys published via DNS
  - Zones signed by parent zones

SNMP Security

- Management Information Base = MIB
- SNMPv1 is simple, effective, and provides the majority of SNMP service in the field
- SNMPv2 adds some functionality to v1
- SNMPv3 is a security overlay for either version, not a standalone replacement

References

[1] https://www.cs.purdue.edu/homes/dec/essay.network.layers.html

[2] http://www.faqs.org/faqs/firewalls-faq/

[3] http://www.sans.org/reading-room/whitepapers/protocols/ssl-tls-beginners-guide-1029

[4] https://www.cs.columbia.edu/~smb/papers/ipext.pdf

[5] http://unixwiz.net/techtips/iguide-kaminsky-dns-vuln.html

[6] D. Chaum. Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms, Communications of the ACM, v. 24, n. 2, Feb. 1981, pages 84-88.

References

[7] http://www.onion-router.net/Publications/IH-1996.pdf

[8] http://www.onion-router.net/Publications/tor-design.pdf

[9] http://avirubin.com/crowds.pdf

[10] http://resources.infosecinstitute.com/ssl-attacks/

[11] https://tools.ietf.org/html/rfc7457

Web Security

Today: Florin Stancu

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY

Computer Science & Engineering Department

Contents

- HTTP Security
  - Cookies, Sessions
  - HTTPS
- Server-side Security
  - Injection
  - Session Hijacking
- Client-side / Browser Security

© Mihai Chiroiu

HTTP Protocol [1]

- Stateless, text-based request-response protocol

Client -> Server:

GET /index.html HTTP/1.0

Header1: value1

Header2: value2

<optional body>



Server -> Client:

HTTP/1.0 200 OK

Header1: value1

Header2: value2

<html><head>...</head>

<body>...</body></html>

© Mihai Chiroiu

HTTP Methods

- **GET**: fetch a resource, may have *query strings*:

  http://domain.com/browse.php?list=users&name=john

  Generates:

  GET /browse.php?list=users&name=john HTTP/1.0

- **PUT / POST**: create or edit a resource (only POST is widely used)
- **DELETE**: delete resources (not used in practice)
- **HEAD**: like GET, but server responds with the headers only
- **OPTIONS**: determine options for a resource
- GET, HEAD and OPTIONS should be idempotent

© Mihai Chiroiu

HTTP Methods & HTML Forms

- Links typically use a **GET** request for opening pages
- HTML forms can generate **GET** and **POST** requests:

```
<form action="/login.php?user_type=regular" method="post">
  User: <input type="text" name="username">
  Password: <input type="password" name="pass">
</form>
```

Generates:

```
POST /login.php?user_type=regular HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 30 <-- the length of the body

username=<username>&pass=<user's password>
```

© Mihai Chiroiu

Cookies

- Small piece of data that the browser stores and sends back to the server on future requests
- Can be used to remember user preferences, server sessions etc.

Response header example:

```
HTTP/1.0 200 OK
Set-Cookie: c1=val1
Set-Cookie: c2=val2
```

Request example:

```
GET / HTTP/1.1
Cookie: cook1=val1;cook2=val2
```

© Mihai Chiroiu

Cookie Security

- Cookies are insecure:
  - The user can freely read & modify them
  - They can be intercepted unless HTTPS is used for transport
- Must add confidentiality and integrity guarantees:
  - Using cryptography: encryption & HMAC [2]
  - Server-side sessions
- Privacy implications:
  - Cookies can be used to track users (e.g. by analytics & ad servers)

© Mihai Chiroiu

Server Sessions

- Also known as server-side cookies
- Server generates a random, unique session ID

  4125a859778b1bf9b9b778a236f01e01

- Server uses database to store secrets associated with a session ID
- Persisted as cookie / passed using GET / POST parameters

  Cookie: **PHPSESSID**=4125a85...

  or

  show.php?**phpsessid**=4125a85...

© Mihai Chiroiu

HTTPS [3]

- Based on Secure Sockets Layer / Transport Layer Security
- Creates a private channel between the client and the server
- The server authenticates itself using certificates and PKI
- Diffie-Hellman for forward secrecy
- Cipher negotiation: RC4, DES, AES CBC, AES GCM etc.
- Target of numerous attacks

© Mihai Chiroiu

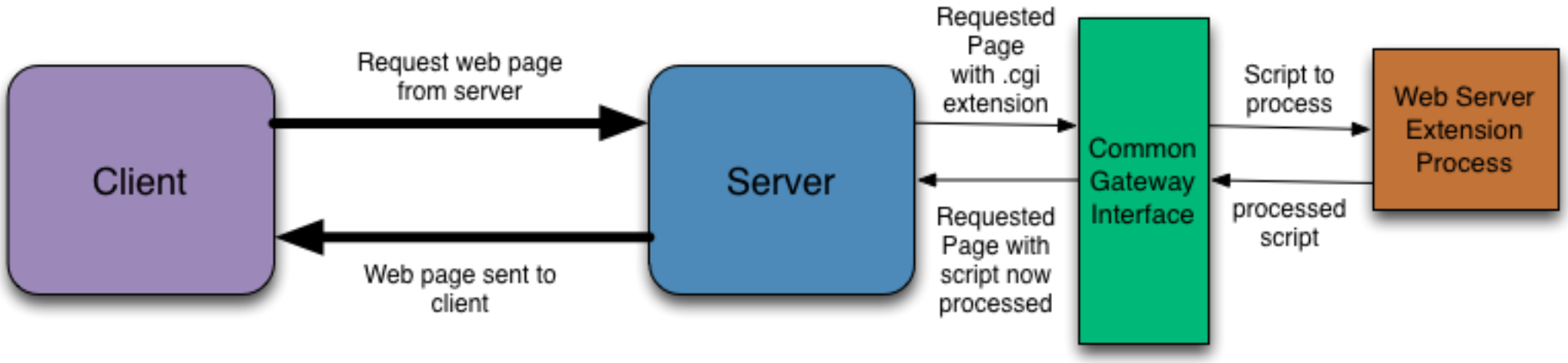
TLS / HTTPS Attacks [4]

- Compression attacks (CRIME, BREACH)
- RC4 weaknesses
- Man-in-the-middle (Malicious Certificates, SSL stripping)
- Downgrade attacks (FREAK, Logjam, POODLE)
- Implementation bugs (e.g. Heartbeat, Cloudflare)

© Mihai Chiroiu

Server-side Processing

- Server generates dynamic content
- Scripting interfaces: CGI (legacy) / FastCGI / apache2 modules [5]



© Mihai Chiroiu

Sample Directory Layout

/var/www

```
  |-- index.html
  |-- login.php
  |-- css/style.css
  |-- images/
      |-- logo.png
      |-- map.png
```

Example requests:

> GET /index.html HTTP/1.0

> GET /images/logo.png HTTP/1.0

> POST /login.php HTTP/1.0

© Mihai Chiroiu

Server-side Processing

- **Example (PHP)**

```
<?php
$name = $_GET["name"];
$curDate = date("l");
?>
<p>Hello, <i><?=$name?></i>.
The date is <b><?=$curDate?></b>
<?php
echo $message;
?>
```

Server-side Injection (1)

SQL Injection [6]

```
$query = "SELECT * FROM users WHERE user='" .
         $_POST["user"] . "' AND password='" .
         hash($_POST["password"]) . "'";
$result = mysql_query($conn, $query);
```

POST: email=admin'--

- => **SELECT * FROM** users **WHERE**
  - user='admin'-- AND password='`

© Mihai Chiroiu

Server-side Injection (2)

- **File upload attack**
- **Example:**
  - A site allows image submissions with minimal verification
  - The user uploads image.gif.php with malicious code
  - User finds out the path to the image and requests it:
    - GET /uploads/image_9876.gif.php
  - Server executes our script (if badly configured ;))

© Mihai Chiroiu

Server-side Injection (3)

- **Preventing injection:**
  - **Do not trust tutorials [7]**
  - **Always sanitize user input!**
  - Try not to use shell execution / script evaluation
  - For SQL, use prepared statements:

```
$stmt = $mysqli->prepare("INSERT INTO table
                          (name) VALUES (?)");
$stmt->bind_param("s", $id); // "s" for string
$stmt->execute();
```

© Mihai Chiroiu

Application-Specific Vectors

- Broken Authentication System [8]
  - Predictable / insecure session IDs
  - Unencrypted passwords [9]
- Authorization Vulnerabilities
  - Improper access verification
  - Example: /delete_user.php?id=5368
  - Direct object reference: /admin/list_users.php
- Vulnerable Frameworks / Plugins (e.g. many Wordpress plugins)

© Mihai Chiroiu

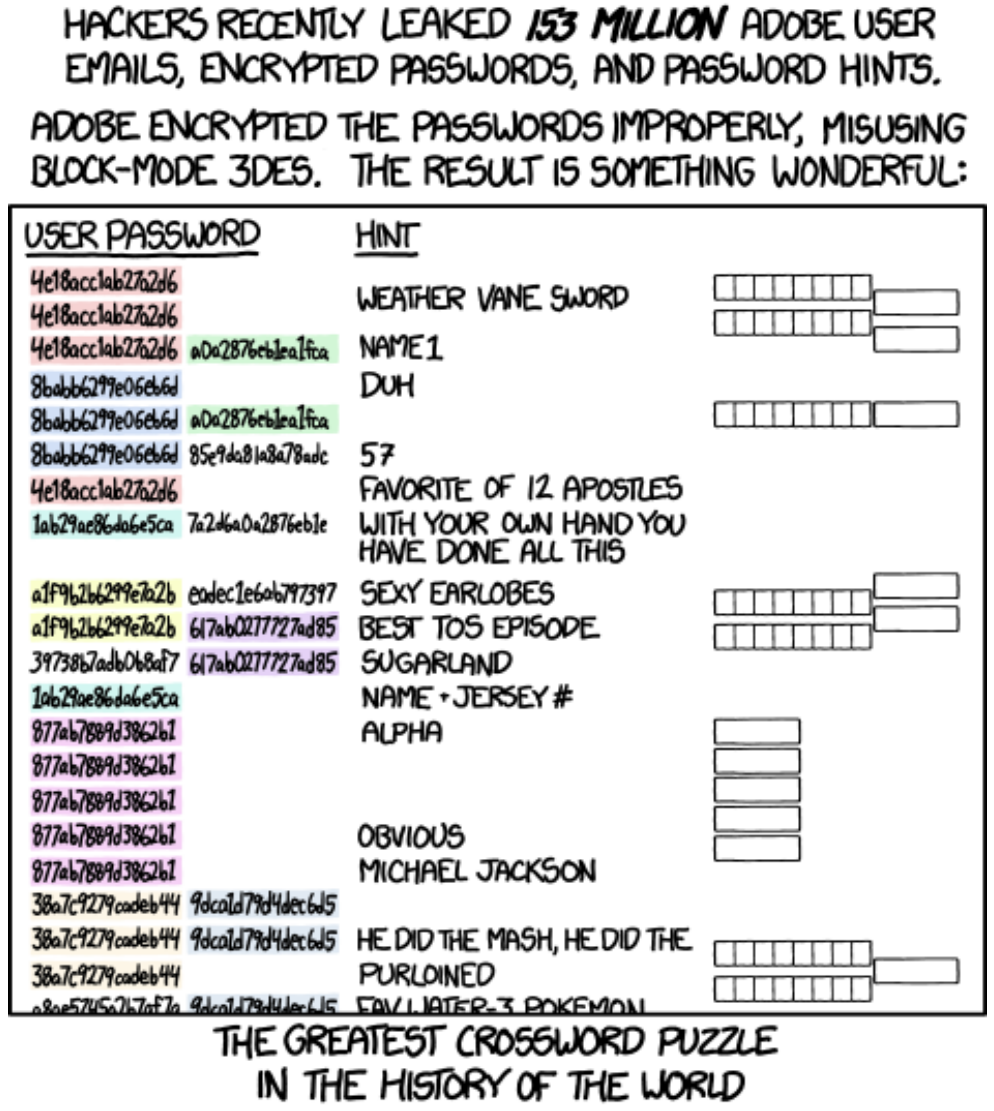
Server Misconfiguration [9]

- **Again: do not trust tutorials**
  - Nginx & PHP FastCGI configuration vulnerability [10]
- Exposed files (e.g. password files, backups) / directory listings
- Bad permissions
- Debugging enabled in production
- System software vulnerabilities:
  - E.g. ShellShock (BASH vulnerability) [11]

© Mihai Chiroiu

Pwned Websites

- **Haveibeenpwned.com** – check it home!
- **Yahoo!** (2012 – SQL Injection, 2013, 2014 – forged cookies)
  - 1 bilion accounts exposed!
- **LinkedIn** (hacked 2012, exposed in 2016)
- **Adobe** (2013): broken encryption =))
- **Dropbox** (2012):
  - SHA1 and salted passwords ;)



© Mihai Chiroiu

SYSTEMS LABORATORY

Computer Science & Engineering Department

Client-side Security

- Client-side Scripting (JavaScript)
  - Isolated execution, resource policies
  - AJAX
- Websites affecting client-side security:
  - Cross-site scripting (XSS)
  - Cross-site request forgery (CSRF)
  - Tracking & Advertisements
- Browser vulnerabilities
- Legacy plugins: ActiveX, Java, Flash

© Mihai Chiroiu

JavaScript

- The most popular ECMAScript implementation [12]
- Used for webpage scripting (dynamic content, animations)
  - Document Object Model
- It can also be used for server scripting (NodeJS)
- Sandboxed execution (e.g. cannot: read user's files, run external programs)
- Modern web applications rendered entirely in JavaScript
  - Angular, React, Polymer etc.

© Mihai Chiroiu

AJAX [13]

- Asynchronous JavaScript and XML
- XMLHttpRequest - API for issuing background HTTP requests
- Used to build modern, responsive applications
- XHR re-sends cookies for the requested domain!

```
var xhr = new XMLHttpRequest();
xhr.open('get', 'ajax.php');
xhr.onreadystatechange = function() {/*...*/};
xhr.send(null);
```

© Mihai Chiroiu

Same / Cross Origin Policies [14]

- Same Origin = Same protocol + domain + port
  - Example: http://domain.com vs https://www.domain.com
- Used to prevent cross-domain data stealing
  - For example, a user enters malicious.com
  - Malicious.com makes a request for facebook.com
  - The request is made, but the response is discarded
- Does not prevent information leakage!
- CORS – Cross-Origin Resource Sharing

© Mihai Chiroiu

CORS

- CORS – Cross-Origin Resource Sharing
- The target server sends special response headers:
  - Access-Control-Allow-Origin: http://domain.com *(for HTTPS)*
  - Origin: http://domain.com *(for HTTP)*
- If the requester's domain matches this ACL, the browser accepts it
- Otherwise, the XHR will receive an error and the response text will be discarded

© Mihai Chiroiu

XSS [15]

- Cross-Site Scripting / client-side code injection
- E.g.: a messaging board website that allows HTML rich text:
  - Someone posts:

    I just wanted to say hello!

    <script>pwnThisSucker();</script>
  - If the target website doesn't filter this, the code will execute on any visitor's browser
- Code can steal data, infect the victims using a browser exploit etc.

© Mihai Chiroiu

XSS Prevention

- Escape HTML before rendering
  - Convert "<" to "<", ">" to ">", quotes to """ etc.
  - Use a template engine that does this
- If rich text is required, use a whitelist-based HTML processor to sanitize
  - Example: strip out dangerous tags like script, embed, iframe etc.
  - **WARNING:** Dont do this unless you know what you're doing!
  - Use a library designed to do this (e.g. htmlpurifier.org)

© Mihai Chiroiu

CSRF [16]

- Cross-Site Request Forgery
- A malicious website tricks the browser / user into accessing a cross-origin URL
- Example (on malicious.com):
  - `<img src="https://www.facebook.com/post/?msg=PWNED!"/>`
- Defenses:
  - Don’t execute critical actions on GET requests!
  - Use CSRF tokens
  - Check headers (Referer, X-Requested-With etc.)

© Mihai Chiroiu

Browser Privacy [17]

- Websites can track the user across multiple domains!
  - Cookies
  - Invisible objects or scripts that do remote requests
  - e.g.: Google Adsense, Google Analytics, Facebook etc.
- Browser Fingerprinting [19]
  - Test yourselves using EFF's Panopticlick [18]
- Tracking servers can become attack vectors!
- Extensions that block such requests [20]

© Mihai Chiroiu

Browser Vulnerabilities

- Browsers are a complex piece of software
- May have vulnerabilities that allow attackers to escape sandboxing
- Attack vectors:
  - Malicious websites
  - Code injection on trusted websites (e.g. XSS)
  - Browser plugins: Flash, Java, ActiveX etc.



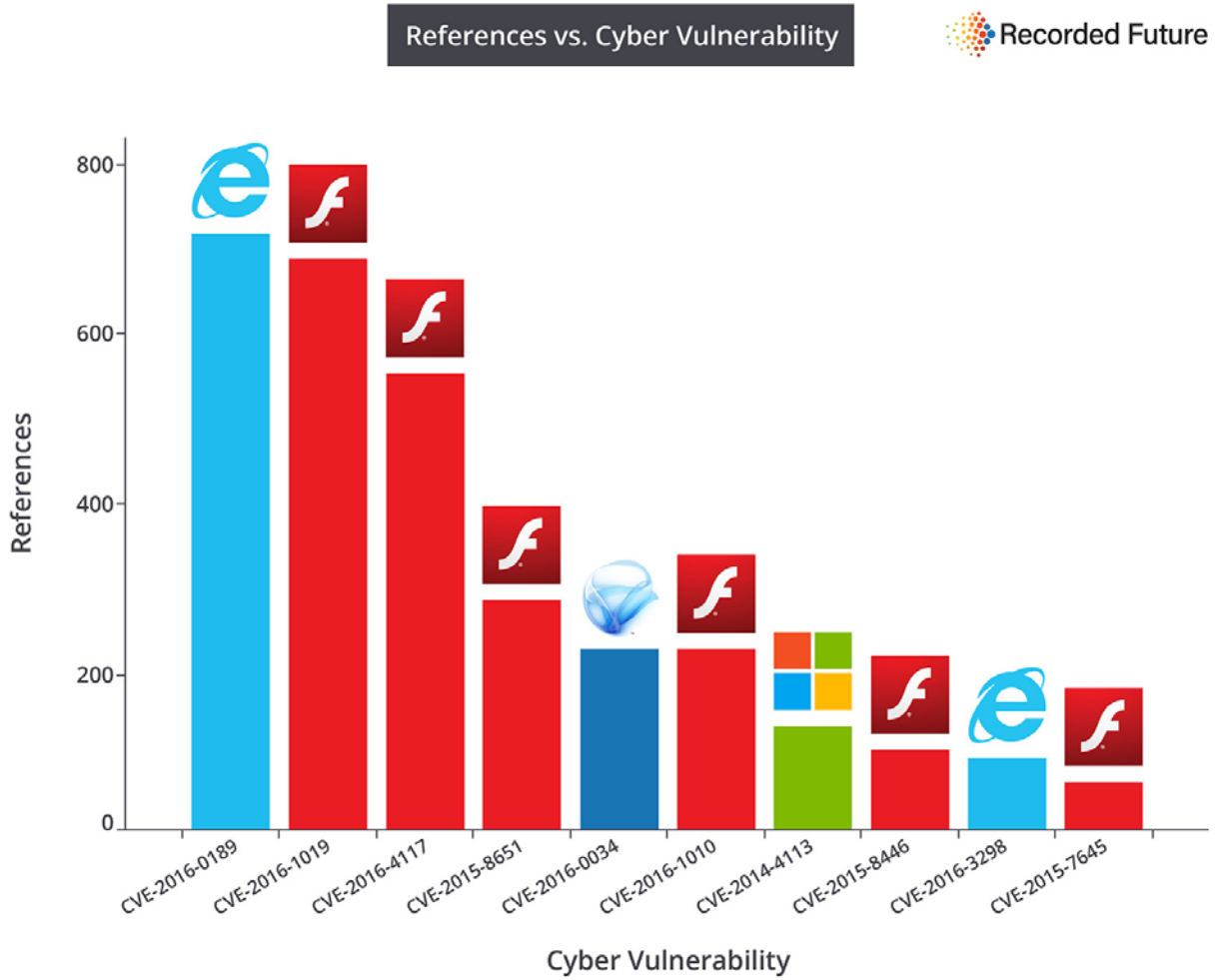




© Mihai Chiroiu

Browser Vulnerabilities (2)

- **2015**: Adobe Flash had 96 vulnerabilities [21]!
- **2016**:
  - Flash most featured in exploit kits!
  - Internet Explorer second place [22]
- **Exploit kits**:
  - Angler, RIG, Nuclear etc.



© Mihai Chiroiu

Browser Vulnerabilities (3)

- **Pwn2Own**: security competition for hacking browsers
- **2016** results [23]:
  - 5 bugs in the Windows operating system
  - 4 bugs in Internet Explorer 11
  - 3 bugs in Mozilla Firefox
  - 3 bugs in Adobe Reader
  - 3 bugs in Adobe Flash
  - 2 bugs in Apple Safari
  - 1 bug in Google Chrome

© Mihai Chiroiu

Secure Browsers

- If you want a secure browser:
  - Don’t use Microsoft’s Internet Explorer!
  - Block all plugins by default
  - Always use the latest version of a browser
- Modern browsers employ multi-process sandboxing
  - One process per tab with no access to the user’s system
  - Coordinate with a main browser process
  - Chromium even uses LXC namespaces on Linux! [24]

© Mihai Chiroiu

OWASP [25]

- The Open Web Application Security Project
- OWASP Top 10 for **2017** (preview [26]):

1. Code Injection
2. Broken Authentication and Session Management
3. Cross-Site Scripting (XSS)
4. Broken Access Control
5. Security Misconfiguration
6. Sensitive Data Exposure
7. Insufficient Attack Protection
8. Cross-site Request Forgery
9. Using components with known vulnerabilities
10. Underprotected APIs

© Mihai Chiroiu

References

[1] HTTP https://tools.ietf.org/html/rfc2616

[2] Murdoch, Steven J. "Hardened stateless session cookies." International Workshop on Security Protocols. Springer Berlin Heidelberg, 2008.

[3] TLS protocol version 1.2, https://tools.ietf.org/html/rfc5246 (2008)

[4] TLS attacks, https://www.rfc-editor.org/rfc/pdfrfc/rfc7457.txt.pdf

[5] Common Gateway Interface, https://tools.ietf.org/html/rfc3875

[6] Clarke-Salt, Justin. SQL injection attacks and defense. Elsevier, 2009.

[6] Flawed Tutorials, https://arxiv.org/pdf/1704.02786.pdf

[7] Session Fixation: http://www.acros.si/papers/session_fixation.pdf

© Mihai Bucicoiu

References (2)

[8] https://fishbowl.pastiche.org/archives/docs/PasswordRecovery.pdf

[9] http://www.pcmag.com/article2/0,2817,11525,00.asp

[10] Common Nginx + PHP Misconfiguration http://bit.ly/1kAK8xu

[11] ShellShock, http://www.securityfocus.com/bid/70103

[12] ECMA-262, http://www.ecma-international.org/publications/standards/Ecma-262.htm

[13] https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest

[14] https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

[15] https://blogs.msdn.microsoft.com/dross/2009/12/15/happy-10th-birthday-cross-site-scripting/

© Mihai Bucicoiu

References (3)

[16] https://www.nccgroup.trust/globalassets/our-research/us/whitepapers/csrf_paper.pdf

[17] http://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=6234427

[18] https://panopticlick.eff.org/

[19] How unique is your browser? https://kabijo.de/files/13/14/5641571611600.pdf

[20] http://lifehacker.com/the-best-browser-extensions-that-protect-your-privacy-479408034

[21] https://heimdalsecurity.com/blog/adobe-flash-vulnerabilities-security-risks/

© Mihai Bucicoiu

References (4)

[22] https://www.recordedfuture.com/top-vulnerabilities-2016/

[23] https://venturebeat.com/2016/03/18/pwn2own-2016-chrome-edge-and-safari-hacked-460k-awarded-in-total/

[24] https://chromium.googlesource.com/chromium/src/+/master/docs/linux_sandboxing.md

[25] https://www.owasp.org/

[26] https://raw.githubusercontent.com/OWASP/Top10/master/2017/OWASP%20Top%2010%20-%202017%20RC1-English.pdf

© Mihai Bucicoiu

Computer Forensics

Asst. Prof. Mihai Chiroiu

SYSTEMS LABORATORY
Computer Science & Engineering Department

Objectives

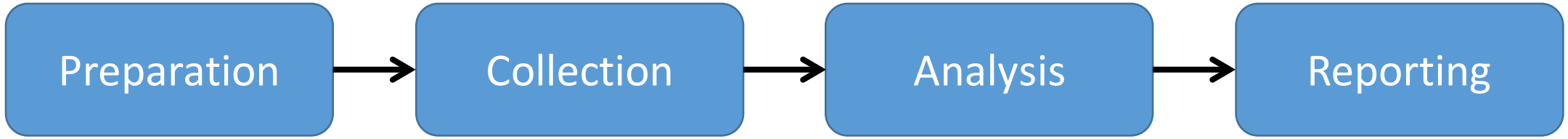
- Digital forensics vocabulary
- Methodology
- Use of investigation tools

Example

- “Prove that email **NOT** was received”
- Easy to prove that an email was received:
  - Look at email servers for logs
- Does not finding traces mean that it was not received?
  - Maybe the investigation was not complete
  - Maybe it was (cleverly) deleted
- Reason for third-party logs ☺

What is Computer Forensics?

- Interpretation of computer media for digital evidence
- Who uses it: Industry, Government, Individual
- Simplified Process Model:



© Mihai Chiroiu

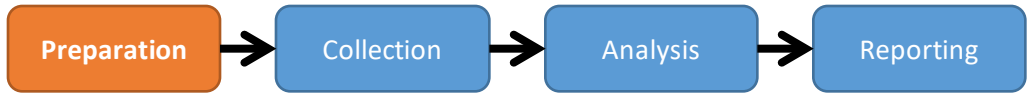
Principles of Computer Forensics

- Anything done to a system changes it!
  - Running systems
- Collection process should retain data integrity
  - How can you prove that you did not change it during the investigation
- All activities should be logged
- Hire an expert or become one ☺

Questions to cover

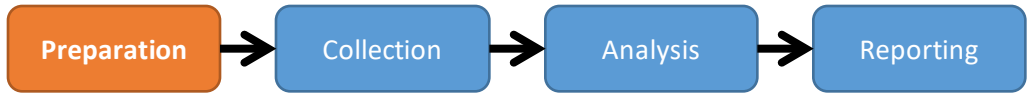
"I keep six honest serving-men
(They taught me all I knew);
Their names are What and Why
and When
And How and Where and Who."

Rudyard Kipling



Questions to cover

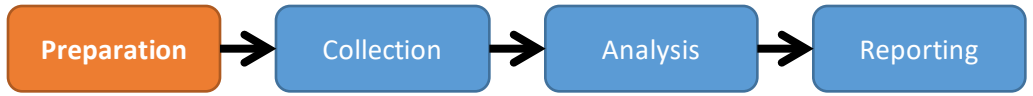
- Has something happened at all?
  - Hacked vs Bugs vs Random effect
- When did it happen and for how long?
- What happened with what effects?
  - Read vs Delete vs Modification
- Who did it and why?
  - IP vs Username
- How did he do it?



Data sources

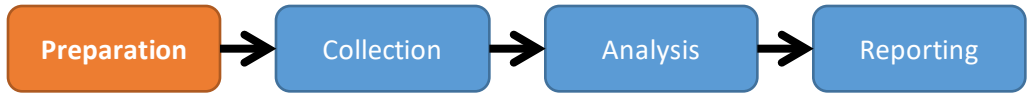
- Make a list of possible data-sourcing
  - Brainstorming
  - e.g., Hard disk, flash drives, mobile phones, scanners, RAM, etc.

- Problem:
  - “Data integrity” -> one must build systems to ensure that it has not been changed
  - e.g., external logging, HSM,



Remote and secure data

- How to investigate the “cloud”?
  - Ask for remote access is not easy, even for governments
- Encrypted disks are hard to investigate
  - Remember TPM?
- Prepare for future investigation
  - IDS-like systems



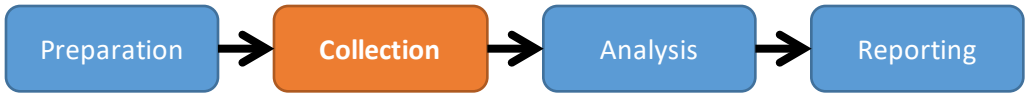
Actions

- Conduct a search for evidence:
  - E.g., Data, Logs, Passwords, Waste paper basket ☺
- Package evidence:
  - When possible, duplicate the systems
    - E.g., make a copy of the hard drive
- Limit access to the collected data
  - Maybe the who is an insider



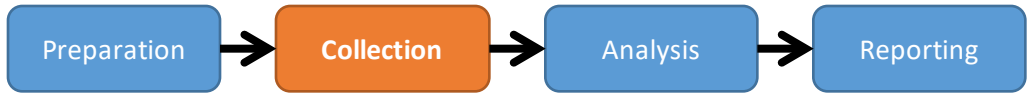
The order of volatility

- Registers, CPU caches
- Routing tables, time-based caches (e.g., arp), kernel statistics
- Running processes
- Memory, Temporary file systems (e.g., ramdisks)
- Media in use
- Remote data (USB drives)
- Backup data, read-only drives (e.g., DVDs)



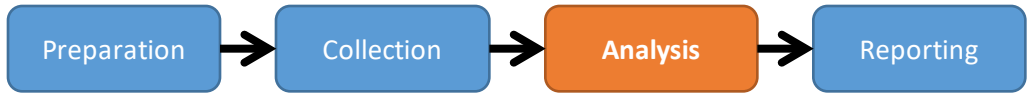
Practical actions for systems

- Show active processes
- List network configuration
- Dump memory (including swap area)
- Stop system
- Duplicate HDD



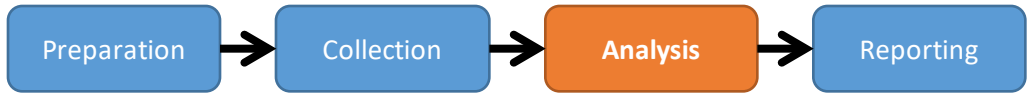
Time difference

- Systems might not have synchronized clocks
- DON’T change the time for investigated systems



Methods of hiding data

- OS-level:
  - Use dot-files, mark as hidden/system
- Change extension/naming: “password.txt” -> “fun.png”
- Unallocated disk sectors
- Steganography (usable for small data, e.g., secrets)
- Encryption
- Fun stuff: encryption + steganography

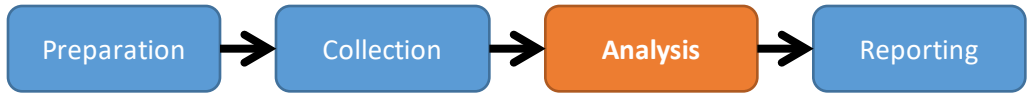


HDD Analysis

- If possible (e.g., magnetic HDD) look at the physical level: platter, interfaces, etc.
- Second, look at low-level partitions, volumes, etc.
- Look at the boot process (ROM-> BIOS->BOOTLOADER->KERNEL)
- Inspect virtualization configuration (if implemented)
- Inspect OS configuration

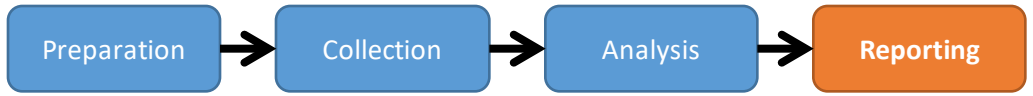
Time difference

- Systems might not have synchronized clocks
- DON’T change the time for investigated systems



Report

- Info of investigator/ Confidence levels
- Identification of case: Date, ID, etc.
- Subject of examination: system, network, etc.
- Summary of findings
- Procedural history: steps taken, detailed findings
- Results and conclusion: server was hacked by X, from Y to Z timeframe, A&B data was accessed; etc.
- Annex



© Mihai Chiroiu

SYSTEMS LABORATORY

Computer Science & Engineering Department

Bibliography

- http://www.fim.uni-linz.ac.at/lva/IT_Recht_Computerforensik/Introduction_to_Computer_Forensics.pdf
- https://www.cs.nmt.edu/~df/